



universidad
de león



Escuela de Ingenierías Industrial, Informática y Aeroespacial

GRADO EN INGENIERÍA DE DATOS E INTELIGENCIA ARTIFICIAL

Trabajo de Fin de Grado

Sistemas de Movilidad Autónoma en Simuladores Urbanos mediante Redes Neuronales y Algoritmos Evolutivos

Autonomous Mobility Systems in Urban Simulators Using Neural Networks and
Evolutionary Algorithms

Autor: Sergio Alves Martín

Tutores: Sergio Rubio Martín / Alicia Merayo Corcoba

(Junio, 2026)

UNIVERSIDAD DE LEÓN
Escuela de Ingenierías Industrial, Informática y
Aeroespacial

GRADO EN INGENIERÍA DE DATOS E
INTELIGENCIA ARTIFICIAL
Trabajo de Fin de Grado

ALUMNO: Sergio Alves Martín

TUTORES: Sergio Rubio Martín / Alicia Merayo Corcoba

TÍTULO: Sistemas de Movilidad Autónoma en Simuladores Urbanos mediante Redes Neuronales y Algoritmos Evolutivos

TITLE: Autonomous Mobility Systems in Urban Simulators Using Neural Networks and Evolutionary Algorithms

CONVOCATORIA: Junio, 2026

RESUMEN:

El trabajo desarrolla un sistema de movilidad autónoma para *CognitiveCarSimulatorReloaded*, simulador urbano en Unreal Engine 5 orientado a rehabilitación y valoración cognitiva con ASPAYM. La solución incorpora vehículos con sensores simulados, interpretación de señales y un controlador neuronal entrenado con algoritmos evolutivos, junto con persistencia de modelos y análisis de métricas CSV. La evaluación principal incluye 548 generaciones de entrenamiento y cuatro sesiones de test con 1134 evaluaciones: fitness máximo de 134959.45 en entrenamiento, 80104.07 en test y una tasa agregada de acierto del 57.50 %. La evolución posterior se documenta como diagnóstico de regresiones, ajuste de señalización y mejora de la instrumentación, no como sustitución automática de esa referencia.

ABSTRACT:

This thesis develops an autonomous mobility system for *CognitiveCarSimulatorReloaded*, an Unreal Engine 5 urban simulator aimed at cognitive rehabilitation and assessment with ASPAYM. The solution adds vehicles with simulated sensors, traffic-sign interpretation and a neural controller trained through evolutionary algorithms, plus model persistence and CSV metric analysis. The main evaluation comprises 548 training generations and four test sessions with 1134 evaluations: maximum fitness values of 134959.45 in training and 80104.07 in testing, with an aggregate success rate of 57.50 %. Later runs are kept as regression, traffic-control and instrumentation diagnostics rather than as an automatic replacement for that reference.

Palabras clave: simulación urbana, movilidad autónoma, redes neuronales, algoritmos evolutivos, Unreal Engine.

Firma del alumno:

VºBº Tutores:

Resumen

CognitiveCarSimulatorReloaded es un simulador urbano desarrollado en Unreal Engine 5 para apoyar actividades de rehabilitación y valoración cognitiva relacionadas con la conducción. Este Trabajo de Fin de Grado aborda una de las carencias detectadas en la versión previa del sistema: la ausencia de tráfico autónomo capaz de generar situaciones dinámicas y repetibles. El objetivo es diseñar e integrar una arquitectura de movilidad vehicular que perciba el entorno, actúe sobre un vehículo con físicas reales, aprenda mediante experimentación y produzca información suficiente para evaluar su comportamiento. La solución combina once sensores distribuidos en un campo de visión de 180 grados, variables dinámicas del vehículo y contexto de semáforos, stops y cedas. Estas señales forman un vector de 20 entradas para un perceptrón multicapa con 16 neuronas ocultas y dos salidas continuas de dirección y aceleración o frenado. La política de control se optimiza mediante un algoritmo evolutivo que conserva modelos históricos y de sesión, muta sus pesos y selecciona individuos con una función de aptitud instrumentada. El sistema incorpora persistencia de modelos, separación entre puntos y trayectorias de entrenamiento y test, validación de señalización y tratamiento específico de rotondas. Cada ejecución exporta métricas a CSV; los scripts de Python verifican su integridad y generan informes y visualizaciones reproducibles.

La evaluación principal utiliza el entrenamiento `train_2026-06-18_11-21`, con 548 generaciones y 16432 registros, y cuatro sesiones posteriores de test que reúnen 1134 evaluaciones sobre 54 puntos de aparición. El mejor fitness alcanza 134959.45 en entrenamiento y 80104.07 en test. Se obtienen 652 aciertos, una tasa del 57.50 % y un intervalo de confianza de Wilson al 95 % entre el 54.60 % y el 60.34 %. El 40.04 % de las evaluaciones termina en colisión y el 2.29 % en infracción normativa; las métricas de rotonda señalan además una recompensa neta negativa y un 33.63 % de colisiones por entrada. Las pruebas posteriores, desde finales de junio hasta el inicio de julio, muestran una evolución más matizada: algunas sesiones conservan parte del rendimiento, otras revelan regresiones por `STOP_VIOLATION` y las últimas incorporan métricas más finas sobre cedas, colas de liberación, bloqueo en cruces, solapes entre vehículos y subtipos de infracción. El proyecto ofrece así una base funcional y medible, pero confirma que cada cambio de recompensa o señalización debe someterse a bloques repetidos antes de considerarse estable.

Abstract

CognitiveCarSimulatorReloaded is an urban driving simulator developed in Unreal Engine 5 to support cognitive rehabilitation and assessment activities related to driving. This Bachelor's Thesis addresses one of the main limitations identified in the previous version of the system: the lack of autonomous traffic capable of producing dynamic and repeatable situations. The objective is to design and integrate a vehicle mobility architecture that perceives its surroundings, controls a physically simulated car, learns through experimentation and produces enough information to evaluate its behaviour. The solution combines eleven sensors distributed across a 180-degree field of view, vehicle dynamics and contextual information from traffic lights, stop signs and yield signs. These signals form a 20-input vector for a multilayer perceptron with 16 hidden neurons and two continuous outputs for steering and acceleration or braking. The evolutionary process preserves historical and session models, mutates their weights and selects individuals through an instrumented fitness function. The system provides model persistence, separate training and test spawn points and routes, traffic-control validation and dedicated roundabout handling. Each run exports CSV metrics that are checked and analysed by reproducible Python scripts.

The main evaluation uses `train_2026-06-18_11-21`, comprising 548 generations and 16432 vehicle records, followed by four test sessions with 1134 evaluations across 54 spawn points. Maximum fitness reaches 134959.45 in training and 80104.07 in testing. The tests produce 652 successes, a 57.50 % success rate and a 95 % Wilson confidence interval from 54.60 % to 60.34 %. Collisions account for 40.04 % of evaluations and traffic-rule violations for 2.29 %; roundabout telemetry also reveals a negative net reward and collisions in 33.63 % of recorded entries. Later runs, from late June to early July, show a more nuanced evolution: some sessions preserve part of the reference performance, others reveal `STOP_VIOLATION` regressions, and the latest diagnostics add finer metrics for yield signs, blocked crossings, vehicle overlaps and violation subtypes. The project therefore provides a functional and measurable baseline while showing that every reward or traffic-control change must pass repeated testing before it can be considered stable.

Índice general

1. Introducción	1
1.1. Organización de la memoria	3
2. Estudio del problema	4
2.1. Contexto	4
2.2. Problema que se resuelve	4
2.3. Objetivo general	5
2.4. Objetivos específicos	5
3. Estado del arte	7
3.1. Simuladores de conducción	7
3.2. Agentes autónomos en entornos urbanos	7
3.3. Redes neuronales y neuroevolución	8
3.4. Retos específicos	8
4. Tecnologías y herramientas	10
4.1. Unreal Engine 5	10
4.2. Sistema de físicas Chaos Vehicle	10
4.3. Raycasting y percepción local	11
4.4. Red neuronal	11
4.5. Algoritmos evolutivos	12
4.6. Análisis de datos	12
4.7. Control de versiones	13
5. Gestión del proyecto	14
5.1. Metodología	14
5.2. Planificación temporal	15
5.3. Seguimiento en GitLab	16
5.4. Cronología de decisiones	18
5.5. Línea evolutiva del desarrollo	19
5.6. Entorno de pruebas inicial	20
5.7. Alcance	21
5.8. Complejidad del cambio de mapa	21
5.9. Gestión de riesgos	22
5.10. Estimación de recursos y presupuesto	23
6. Núcleo del trabajo	24

6.1.	Descripción general de la solución	24
6.2.	Subsistemas	24
6.2.1.	Gestor evolutivo	24
6.2.2.	Transición a coches físicos no fantasma	25
6.2.3.	Controlador de IA y red neuronal	26
6.2.4.	Percepción mediante sensores	27
6.2.5.	Sistema de fitness	28
6.2.6.	Semáforos	31
6.2.7.	Stop y ceda el paso	31
6.2.8.	Rotondas	32
6.3.	Pipeline experimental	32
7.	Resultados	34
7.1.	Datos analizados	34
7.2.	Resumen cuantitativo	35
7.3.	Evolución del entrenamiento	35
7.4.	Evaluación de test	37
7.5.	Razones de finalización	38
7.6.	Comportamiento ante señalización	39
7.7.	Comportamiento en rotondas	39
7.8.	Dificultad por puntos de aparición	40
7.9.	Pruebas de regresión y diagnóstico	40
8.	Normativa y accesibilidad	42
8.1.	Protección de datos	42
8.2.	Seguridad y uso responsable	42
8.3.	Licencias de assets externos	43
8.4.	Accesibilidad	43
9.	Conclusiones y líneas futuras	45
9.1.	Conclusiones	45
9.2.	Aportaciones realizadas	46
9.3.	Problemas encontrados	47
9.4.	Líneas futuras	48
	Agradecimientos	50
	Bibliografía	51
A.	Fuentes analizadas	A-1
A.1.	Documentación administrativa y de formato	A-1
A.2.	Actas de reunión	A-1
A.3.	Seguimiento en GitLab	A-2

A.4.	Descripciones del programa	A-2
A.5.	Resultados experimentales	A-2
A.6.	Conversaciones con asistentes de IA	A-3
B.	Prompts utilizados	B-1
B.1.	Criterio de documentación	B-1
B.2.	Material revisado	B-1
B.3.	Prompts depurados incorporados	B-2

Índice de figuras

Figura 7.1. Tendencia del fitness en el entrenamiento del 18 de junio de 2026. 37

Índice de tablas

Tabla 1.1.	Visión inicial del sistema desarrollado.	2
Tabla 5.1.	Diagrama de Gantt simplificado del desarrollo.	15
Tabla 5.2.	Milestones e issues cerradas registradas en GitLab.	16
Tabla 5.3.	Principales hitos y decisiones documentadas en reuniones.	18
Tabla 5.4.	Línea evolutiva del desarrollo técnico.	19
Tabla 5.5.	Riesgos principales del proyecto.	22
Tabla 6.1.	Lectura práctica de los cálculos principales del fitness.	29
Tabla 7.1.	Resultados principales del entrenamiento y del bloque de test del 18 de junio de 2026.	35
Tabla 7.2.	Evolución de la red neuronal y del fitness durante el proyecto. . . .	36
Tabla 7.3.	Evolución de los bloques de test archivados entre junio y julio de 2026.	38
Tabla B.1.	Conversaciones de IA revisadas para el anexo de prompts.	B-1

Glosario

ASPAYM	Asociación/Fundación colaboradora orientada a mejorar la calidad de vida de personas con lesión medular, gran discapacidad física y daño cerebral adquirido.
Blueprint	Sistema visual de programación de Unreal Engine empleado para implementar lógica de juego, interacción, controladores y sistemas de simulación.
Chaos Vehicle	Sistema de físicas de Unreal Engine 5 utilizado para simular el comportamiento dinámico del vehículo.
DCA	Daño Cerebral Adquirido.
Fitness	Función de aptitud que cuantifica el comportamiento de cada vehículo autónomo durante una generación evolutiva.
NavMesh	Malla de navegación utilizada por motores de videojuegos para calcular rutas transitables. En este proyecto fue estudiada y posteriormente descartada como solución principal para vehículos.
NPC	<i>Non-Player Character</i> ; agente controlado por el sistema y no por el usuario.
Raycasting	Técnica de consulta geométrica que lanza rayos o trazas desde un actor para detectar obstáculos y distancias en el entorno.
RN	Red neuronal artificial.
UE5	Unreal Engine 5.

Capítulo 1

Introducción

Este Trabajo de Fin de Grado se enmarca en la evolución de *CognitiveCarSimulatorReloaded*, un simulador urbano de conducción desarrollado en Unreal Engine 5 y orientado a actividades de rehabilitación y valoración cognitiva. El simulador ya disponía de una ciudad funcional, un vehículo conducible, señalización y registro de datos, pero todavía carecía de un elemento clave para aproximarse a una situación de tráfico real: la presencia de vehículos autónomos capaces de circular, detenerse, reaccionar ante señales y generar eventos dinámicos alrededor del usuario.

La motivación del proyecto nace precisamente de esa carencia. Un escenario urbano vacío permite practicar el manejo básico, pero no reproduce la carga cognitiva que aparece cuando hay otros agentes en movimiento, decisiones de prioridad, esperas, cruces, rotondas o situaciones imprevistas. Para un simulador que aspira a servir como apoyo a profesionales y usuarios en contextos de rehabilitación, el tráfico autónomo no es un adorno visual: es una herramienta para construir situaciones repetibles, graduables y medibles.

La solución desarrollada introduce vehículos controlados por una red neuronal ligera, entrenada mediante un algoritmo evolutivo dentro del propio simulador. Cada coche percibe el entorno mediante sensores simulados con raycasting, recibe contexto de semáforos, stops, cedas e intersecciones, y transforma esas entradas en dos acciones continuas: dirección y aceleración o frenado. El controlador no teletransporta ni desplaza el coche por una ruta prefijada, sino que aplica esas acciones sobre el sistema físico de Unreal Engine. Esta decisión hace el problema más difícil, porque aparecen inercia, radio de giro, frenada, colisiones y pérdida de control, pero también lo acerca más al objetivo real del simulador.

El desarrollo no siguió una línea directa desde la primera implementación hasta el resultado final. En las fases iniciales se probaron enfoques deterministas de navegación, como rutas y *Path Finding*, que sirvieron para entender el mapa pero produjeron comportamientos rígidos en un vehículo físico. Después se construyó una línea de pruebas reducida para aprender centrado, avance y giros básicos. Más adelante se introdujeron puntos de aparición distribuidos por la ciudad, separación entre entrenamiento y test,

señales de tráfico, intersecciones, rotondas, persistencia de modelos y análisis sistemático de CSV. La última fase añadió además interacción física entre coches, detección de solapes, distancia de seguridad y un modo de simulación libre.

Por tanto, esta memoria no presenta únicamente el estado final del programa. También reconstruye la evolución técnica que llevó hasta él: qué se intentó, qué falló, qué hubo que instrumentar y qué decisiones cambiaron el rumbo del proyecto. Esta lectura es importante porque el sistema combina aprendizaje automático, físicas en tiempo real, configuración manual del mapa y reglas de tráfico. Una mejora local en el fitness, en un stop o en una rotonda puede provocar una regresión en otra parte del escenario; por eso los resultados deben interpretarse como una base funcional y medible, no como una solución cerrada lista para validación clínica.

Tabla 1.1. Visión inicial del sistema desarrollado.

Elemento	Papel dentro del TFG
Simulador base	Ciudad urbana en Unreal Engine 5 heredada de <i>CognitiveCar-SimulatorReloaded</i> , utilizada como entorno físico y visual de experimentación.
Agente autónomo	Vehículo controlado por red neuronal que decide dirección y aceleración/frenado a partir de sensores, estado dinámico y contexto de tráfico.
Aprendizaje	Gestor evolutivo que evalúa poblaciones, conserva modelos destacados, muta pesos y separa entrenamiento, test y simulación libre.
Entorno de prueba	Progresión desde una línea inicial controlada hasta puntos de aparición distribuidos por todo el mapa, con trayectorias de entrenamiento y test diferenciadas.
Validación	Registros CSV, análisis reproducible en Python, tasas de acierto, familias de fallo, métricas por spawn, señalización, rotondas y solapes entre vehículos.

Fuente. *Elaboración propia a partir del código, las descripciones fechadas de Blueprints y los CSV experimentales.*

La evaluación principal utiliza el entrenamiento del 18 de junio de 2026 y cuatro sesiones de test posteriores sobre 54 puntos de aparición. Ese bloque obtiene una tasa de acierto del 57.50 %, con 652 aciertos sobre 1134 evaluaciones. La serie posterior, desde finales de junio hasta el 3 de julio, se conserva como diagnóstico: algunas pruebas mantienen parte del comportamiento aprendido, otras revelan regresiones de señalización y las últimas amplían la observación de cedas, stops, solapes, colas de liberación y bloqueo en cruces. Esta separación evita presentar la fecha más reciente como la mejor por defecto y permite distinguir una referencia estable de una iteración exploratoria.

1.1 ORGANIZACIÓN DE LA MEMORIA

La lectura del documento sigue el mismo recorrido que siguió el proyecto. Primero se delimita la necesidad que motiva el trabajo; después se justifican las alternativas técnicas; a continuación se describe cómo se planificó, implementó y evaluó la solución; por último se interpretan los resultados y se señalan los límites que todavía impiden considerar el sistema cerrado. Esta organización busca que el lector no vea la red neuronal como un componente aislado, sino como una pieza dentro de un simulador con físicas, mapa, señalización, registros y restricciones de uso.

También se separan de forma explícita tres niveles de evidencia. El primero es el código y la configuración de Blueprints, que explican qué hace el sistema. El segundo son las actas, issues y descripciones fechadas, que permiten reconstruir por qué se tomaron determinadas decisiones. El tercero son los CSV experimentales, que muestran qué ocurrió al ejecutar el simulador. Mantener esos niveles separados ayuda a distinguir una intención de diseño, una modificación implementada y un resultado medido.

El documento se organiza en nueve capítulos principales. Tras esta introducción, el capítulo 2 estudia el problema que se quiere resolver y concreta los objetivos del trabajo. El capítulo 3 resume el estado del arte y los retos asociados a simuladores, agentes autónomos y aprendizaje evolutivo. El capítulo 4 describe las tecnologías y herramientas utilizadas. El capítulo 5 presenta la gestión del proyecto, la evolución temporal, los riesgos y una estimación preliminar de recursos. El capítulo 6 desarrolla el núcleo técnico de la solución. El capítulo 7 recoge los resultados experimentales. El capítulo 8 trata normativa, protección de datos, licencias de assets y accesibilidad. Finalmente, el capítulo 9 expone conclusiones, aportaciones y líneas futuras.

Capítulo 2

Estudio del problema

2.1 CONTEXTO

La conducción es una actividad compleja que exige coordinación motora, atención sostenida, atención dividida, interpretación de señales, percepción espacial y toma de decisiones bajo restricciones temporales. En personas que han sufrido daño cerebral adquirido, estas capacidades pueden verse alteradas aunque existan adaptaciones físicas que permitan manejar un vehículo. Por ello, antes de exponer al paciente a situaciones reales de tráfico, resulta de interés disponer de entornos controlados donde observar, entrenar y valorar comportamientos de conducción de forma segura.

El proyecto *CognitiveCarSimulatorReloaded* nace dentro de esta necesidad. La propuesta original define un simulador realista de conducción urbana para entrenamiento y valoración de alteraciones cognitivas tras un daño cerebral adquirido, desarrollado en colaboración con ASPAYM [1]. La versión inicial del simulador, desarrollada durante el curso 2024-2025, ya incluía un entorno urbano, vehículo, interacción física, señalización y registro de datos [2]. Sin embargo, las evaluaciones posteriores identificaron una carencia clave: el escenario necesitaba tráfico autónomo y dinámico para trasladar al usuario a una situación más realista.

El presente TFG aborda esa carencia desde el ámbito de la Ingeniería de Datos e Inteligencia Artificial. El foco no se sitúa en crear un simulador desde cero, sino en diseñar e integrar modelos de movilidad autónoma capaces de aprender comportamientos de conducción dentro de un entorno urbano ya existente. El trabajo combina redes neuronales, algoritmos evolutivos, sensores simulados, control físico en tiempo real y análisis de datos experimentales.

2.2 PROBLEMA QUE SE RESUELVE

El problema técnico principal consiste en conseguir que agentes autónomos, inicialmente vehículos, circulen por un mapa urbano de forma suficientemente estable, medible y compatible con las normas básicas de tráfico. A diferencia de un recorrido prefijado, un agente autónomo debe responder a múltiples condiciones locales: límites

de la calzada, obstáculos, intersecciones, semáforos, señales de stop, señales de ceda el paso y cambios en su propio estado dinámico.

Durante el desarrollo se evaluaron soluciones deterministas de navegación, como el uso de NavMesh o recorridos mediante *Path Finding*. Estas aproximaciones resultaron útiles como punto de partida, pero presentaron limitaciones para el objetivo clínico y experimental del simulador: rutas rígidas, giros poco naturales, dificultad para representar la toma de decisiones local y menor capacidad de adaptación a situaciones imprevistas. Como consecuencia, el proyecto evolucionó hacia una solución basada en percepción local mediante sensores y aprendizaje evolutivo.

Además, el problema requiere que el comportamiento obtenido sea observable y explicable. No basta con que los vehículos se desplacen por el escenario; es necesario registrar por qué avanzan, se detienen, colisionan, incumplen una señal o completan correctamente una maniobra. Esta trazabilidad condiciona la arquitectura de la solución, porque cada decisión de control debe poder relacionarse con sensores, contexto de tráfico, componentes de fitness y resultados experimentales.

2.3 OBJETIVO GENERAL

El objetivo general del trabajo es desarrollar e integrar un sistema de movilidad autónoma para un simulador urbano de conducción, basado en redes neuronales y algoritmos evolutivos, que permita entrenar, evaluar y analizar vehículos autónomos dentro de Unreal Engine 5.

De forma complementaria, el proyecto busca dejar una base reutilizable para futuras ampliaciones del simulador. La movilidad vehicular autónoma actúa como primer bloque técnico sobre el que podrán incorporarse perfiles de conducción, otros agentes dinámicos y escenarios de validación más cercanos al uso clínico previsto.

2.4 OBJETIVOS ESPECÍFICOS

El objetivo general se concreta en tareas que cubren el ciclo completo de desarrollo: conocer el punto de partida, construir el controlador, entrenarlo y comprobar su comportamiento con datos. No se plantea únicamente obtener vehículos que se desplacen por el mapa; también se busca que las decisiones de diseño queden justificadas y que los resultados puedan repetirse y compararse. A partir de este criterio, los objetivos específicos son:

- Analizar el estado inicial del simulador y las restricciones técnicas heredadas de la versión previa.
- Diseñar una arquitectura de percepción para vehículos autónomos basada en sensores simulados mediante raycasting.

- Implementar una red neuronal ligera capaz de transformar entradas sensoriales y de contexto en acciones de conducción.
- Diseñar un proceso de entrenamiento evolutivo que evalúe poblaciones de vehículos, seleccione individuos destacados y aplique mutaciones sobre sus pesos.
- Integrar señales de tráfico, semáforos, stops y cedas en la lógica de percepción, parada y validación de la IA.
- Construir una función de fitness interpretable, con componentes positivos y penalizaciones, que permita medir progreso, errores y comportamientos no deseados.
- Generar registros experimentales en CSV y analizarlos mediante scripts reproducibles para evaluar entrenamiento y test.
- Identificar limitaciones, riesgos y líneas futuras para ampliar la movilidad autónoma a perfiles de conductor, peatones, bicicletas y otros agentes.

Capítulo 3

Estado del arte

3.1 SIMULADORES DE CONDUCCIÓN

Los simuladores de conducción permiten reproducir situaciones de tráfico en un entorno controlado, repetible y seguro. En el ámbito formativo se utilizan para entrenar maniobras, respuesta ante eventos inesperados y familiarización con normas de circulación. En el ámbito sanitario, su valor reside en que permiten observar capacidades cognitivas y motoras sin exponer al usuario a riesgos reales. La versión previa de *CognitiveCarSimulator* ya se apoyaba en esta idea: un entorno urbano en Unreal Engine donde registrar tiempos de reacción, velocidades, colisiones y datos exportables para especialistas [2].

Para que un simulador de rehabilitación resulte convincente, el entorno no puede limitarse a una vía vacía. El usuario debe compartir la escena con otros agentes que creen carga cognitiva: vehículos que se detienen, peatones que cruzan, ciclistas, señales y elementos dinámicos. La propuesta de *CognitiveCarSimulatorReloaded* identifica precisamente esa necesidad al plantear IAs conductoras configurables, peatones, bicicletas y animales como ampliaciones progresivas del simulador [1].

3.2 AGENTES AUTÓNOMOS EN ENTORNOS URBANOS

La navegación autónoma en videojuegos y simulación suele abordarse mediante técnicas deterministas, como grafos de rutas, splines o mallas de navegación. Estas técnicas son robustas cuando el objetivo es desplazar agentes por zonas transitables con trayectorias controladas, pero pueden producir comportamientos rígidos si se aplican directamente a vehículos físicos. Un coche no se desplaza como un personaje humano: tiene inercia, radio de giro, aceleración, frenada, fricción y restricciones de control continuas.

En este proyecto se comprobó esa diferencia durante las primeras iteraciones. El uso de NavMesh y Path Finding fue estudiado para permitir recorridos por la ciudad, pero las reuniones de seguimiento y los informes técnicos documentan problemas de rigidez, dificultad en intersecciones y falta de naturalidad. Por ello se adoptó una estrategia más cercana a la percepción local: el agente no sigue una ruta global cerrada, sino que interpreta distancias, estado del tráfico y contexto normativo para producir acciones continuas.

3.3 REDES NEURONALES Y NEUROEVOLUCIÓN

Las redes neuronales artificiales permiten aproximar funciones complejas a partir de entradas numéricas [9]. En este TFG se emplean como política de control: reciben sensores, velocidad y contexto de señalización, y producen dirección y aceleración/frenado. La configuración vigente es deliberadamente ligera para poder ejecutarse en tiempo real sobre múltiples vehículos: 20 entradas, una capa oculta de 16 neuronas y dos salidas continuas con activación hiperbólica.

El entrenamiento no se plantea como aprendizaje supervisado, ya que no se dispone de un conjunto amplio de trayectorias humanas etiquetadas dentro del simulador. En su lugar, se adopta un algoritmo evolutivo [10]. Cada individuo de una población representa un conjunto de pesos de la red neuronal. La simulación evalúa el comportamiento mediante fitness; los mejores pesos se conservan, se mutan y se reutilizan en generaciones posteriores. Este enfoque se sitúa dentro de la neuroevolución, aunque el proyecto mantiene fija la topología de la red y evoluciona únicamente sus pesos, a diferencia de métodos como NEAT que también modifican la estructura [11].

La elección de un enfoque evolutivo también facilita trabajar con objetivos parcialmente contradictorios. Un vehículo debe avanzar, pero no a cualquier precio; debe mantener velocidad útil, pero frenar ante obstáculos; debe explorar la ciudad, pero respetar stops, cedas y semáforos. En este contexto, el fitness actúa como mecanismo de equilibrio entre progreso, seguridad, cumplimiento normativo y estabilidad de la conducción.

3.4 RETOS ESPECÍFICOS

La revisión de las alternativas disponibles y las primeras pruebas sobre el simulador permitieron acotar los problemas que condicionaban la solución. Algunos proceden del propio vehículo físico y otros aparecen al trasladar un método de aprendizaje a un mapa urbano con cruces y señalización. Los principales retos identificados son:

- **Simulación física:** el control neuronal debe traducirse a entradas de un vehículo con físicas reales, evitando aceleraciones, frenadas o giros no plausibles.

- **Intersecciones:** los cruces concentran la mayor complejidad, porque combinan decisión de trayectoria, detección de límites, prioridades y otros vehículos.
- **Diseño de fitness:** una función de aptitud mal calibrada puede premiar estrategias no deseadas, como detenerse indefinidamente, avanzar demasiado despacio o compensar errores graves con comportamientos parciales.
- **Generalización por puntos de aparición:** el modelo debe funcionar en diferentes zonas del mapa, no únicamente en los puntos de entrenamiento más sencillos.
- **Coste computacional:** entrenar decenas de vehículos simultáneamente en Unreal Engine implica carga de CPU, GPU y física.
- **Trazabilidad:** al tratarse de un sistema experimental, es imprescindible registrar métricas suficientes para explicar por qué una iteración mejora o empeora.

Estos retos aparecen de forma combinada durante el desarrollo. Una mejora en sensores puede modificar el fitness, un cambio en intersecciones puede alterar la tasa de colisiones y una reducción de coste computacional puede afectar al número de generaciones evaluables. Por ello, el proyecto no se aborda como una única implementación cerrada, sino como un proceso de ajuste continuo apoyado en métricas y revisión experimental.

Capítulo 4

Tecnologías y herramientas

4.1 UNREAL ENGINE 5

Unreal Engine 5 es el motor sobre el que se desarrolla el simulador. Proporciona renderizado 3D, editor visual, sistema de actores, colisiones, físicas, Blueprints y herramientas de depuración [3, 5]. Su elección viene heredada de la versión inicial del proyecto y resulta coherente con los requisitos de realismo visual, interacción en tiempo real y despliegue del simulador como aplicación ejecutable.

El trabajo se implementa principalmente mediante Blueprints, lo que permite integrar lógica de IA, controladores, sensores y eventos de tráfico dentro del propio editor. Este enfoque facilita la iteración rápida, aunque introduce retos de mantenibilidad cuando los grafos crecen. Por ello se han documentado los Blueprints principales del gestor evolutivo, el controlador de IA, el vehículo autónomo y los actores de señalización. Las denominaciones concretas de estos módulos se recogen en el anexo de fuentes para mantener la trazabilidad técnica sin sobrecargar el texto principal.

4.2 SISTEMA DE FÍSICAS CHAOS VEHICLE

El vehículo se controla mediante el sistema de físicas Chaos Vehicles de Unreal Engine [4]. La red neuronal no mueve directamente el actor, sino que genera consignas continuas que se aplican al componente de movimiento del vehículo. La salida de dirección se envía como *steering input*; la salida de aceleración/frenado se interpreta de forma separada: valores positivos se aplican como acelerador y valores negativos como freno.

Esta separación es importante porque mantiene el comportamiento dentro de la física del motor. El agente aprende a actuar sobre el vehículo, pero la respuesta final depende de velocidad, fricción, inercia, contacto con el suelo y restricciones propias del modelo físico.

4.3 RAYCASTING Y PERCEPCIÓN LOCAL

La percepción del vehículo se implementa mediante trazas o rayos simulados, una técnica soportada por Unreal Engine para consultar colisiones a lo largo de una línea de la escena [6]. El controlador lanza once sensores en un ángulo de visión de 180 grados desde el vehículo. Cada sensor devuelve una distancia normalizada al primer obstáculo relevante, con valor cercano a 1 cuando no se detecta un obstáculo próximo y valores menores cuando existe un límite, pared, vehículo o borde de intersección.

Además de los sensores laterales y frontales, se calculan variables derivadas como la distancia frontal a obstáculo, la distancia del sensor izquierdo, la distancia del sensor derecho y la diferencia horizontal entre ambos. Estas magnitudes permiten construir una señal de centrado en carril y detectar correcciones equivocadas.

4.4 RED NEURONAL

La red neuronal empleada es un perceptrón multicapa ligero. Las primeras trece posiciones de su vector de entrada describen el movimiento y el espacio próximo al vehículo: velocidad longitudinal, velocidad angular y once distancias obtenidas mediante raycasting. Todas ellas se normalizan antes de ejecutar la inferencia para evitar que una magnitud domine a las demás únicamente por su escala.

Las siete posiciones restantes incorporan el contexto de señalización y el sesgo. Dos contienen el estado numérico del control activo y la distancia normalizada a la línea de parada; esta última solo se utiliza cuando hay un semáforo activo o una orden de parada, y vale 1 en el resto de situaciones. Otras cuatro posiciones forman una codificación *one-hot* que distingue entre semáforo, stop, ceda el paso o ausencia de control. La última posición es un sesgo constante. La configuración vigente suma así 20 entradas y permite que una misma lectura geométrica produzca respuestas distintas según la norma aplicable.

La capa oculta contiene 16 neuronas y utiliza $\tanh$ como función de activación. La capa de salida contiene dos neuronas, también con $\tanh$, de modo que sus valores se encuentran en el rango $[-1, 1]$. La primera salida controla dirección y la segunda controla aceleración o frenado. La arquitectura emplea 320 pesos entre entrada y capa oculta y 32 entre capa oculta y salida, 352 en total. La rutina de inferencia deduce el número esperado de entradas a partir de la longitud del primer bloque de pesos y comprueba ambas dimensiones antes de calcular las salidas. Esta guarda permite rechazar modelos incompatibles si la configuración cambia.

4.5 ALGORITMOS EVOLUTIVOS

El entrenamiento se basa en población, selección y mutación. El gestor evolutivo crea una población de vehículos, les asigna pesos iniciales o mutados, ejecuta la simulación y registra el fitness final de cada individuo. Los mejores pesos se guardan en disco como modelos reutilizables, distinguiendo entre el mejor modelo histórico y el mejor de sesión.

La configuración documentada el 17 de junio aplica una tasa ordinaria del 3 % a los descendientes del modelo histórico y del mejor modelo de sesión. Cada generación conserva además una copia sin mutar de ambos modelos y reserva el resto de la población a una mutación del 6 %, más intensa. El reparto equilibra explotación y diversidad sin reinicializar aleatoriamente individuos después de la primera generación. Los tamaños de población son parámetros editables del mapa; por ello, los valores efectivos se obtienen de la columna `N` de cada ejecución y no del valor por defecto del Blueprint.

4.6 ANÁLISIS DE DATOS

La instrumentación se diseñó para separar el detalle de cada vehículo de los resúmenes utilizados al comparar generaciones. Esta separación evita que los ficheros agregados crezcan innecesariamente y, al mismo tiempo, permite volver al registro individual cuando aparece una colisión, una penalización o un resultado anómalo. El proyecto genera tres ficheros principales:

- `Fitness_Debug.csv`, con métricas por coche: fitness, tiempo de vida, punto de aparición, razón de muerte, penalizaciones, bonus, datos de conducción, validación de stops y cedas, mutaciones, genealogía y comportamiento en rotondas.
- `Training_Summary.csv`, con resumen por generación de entrenamiento.
- `Test_Summary.csv`, con resumen por generación de test, incluyendo aciertos, errores y tasas.

El análisis posterior se realiza mediante Python, NumPy, Matplotlib y Seaborn. El script `analyse_current.py` valida la calidad de los CSV, detecta automáticamente si la sesión es de entrenamiento o test, genera informes textuales y produce gráficas de evolución, muertes, distribución de fitness, ranking por spawn, control de entradas, convergencia, diversidad, correlaciones, solapes entre coches y diagnósticos de paradas, cedas, bloqueo en cruces y rotondas. La versión actual calcula las métricas situacionales solo sobre filas con exposición al evento correspondiente, evitando que los ceros de coches que nunca alcanzaron una rotonda diluyan los promedios. El script `clean_unreal_log.py` permite limpiar logs de Unreal Engine conservando errores, avisos relevantes, mensajes de Blueprint y trazas propias del proyecto.

4.7 CONTROL DE VERSIONES

El trabajo se ha desarrollado con Git/GitLab en un contexto colaborativo. La documentación de seguimiento recoge tareas de limpieza de repositorio, configuración de `.gitignore`, eliminación de binarios pesados y organización de issues y milestones. El registro depurado utilizado en esta memoria contiene 40 issues cerradas distribuidas en 15 sprints, lo que permite reconstruir la evolución desde la reorganización inicial del proyecto hasta las últimas mejoras de cedas, solapes y rotondas. Esta disciplina ha sido relevante porque el proyecto combina assets de Unreal, Blueprints, datos generados y modelos guardados.

El uso de control de versiones también condicionó la forma de documentar el trabajo. Los cambios de comportamiento no siempre quedaban reflejados en una única clase textual, ya que una parte importante de la lógica vive en Blueprints y assets binarios. Por ello, las issues y milestones se usaron como una capa descriptiva complementaria: cada bloque de trabajo resume el objetivo, la fecha y la consecuencia técnica, mientras que las carpetas de `INFO` conservan descripciones exportadas y análisis de resultados. Esta combinación permite auditar el proceso sin depender solo del historial de commits ni de capturas manuales del editor.

Capítulo 5

Gestión del proyecto

5.1 METODOLOGÍA

El desarrollo se ha organizado mediante un enfoque iterativo e incremental. Las reuniones de seguimiento se celebraron de forma periódica, generalmente cada dos jueves, con participación de tutores, equipo técnico y personas vinculadas al proyecto original. En estas reuniones se revisaban avances, problemas, decisiones de arquitectura y siguientes objetivos. Esta dinámica se aproxima a una metodología ágil por sprints, aunque adaptada al contexto académico y a la disponibilidad del equipo.

La planificación inicial contemplaba una progresión amplia: semáforos y señales, IA vehicular, stops y cedas, perfiles de conductor, peatones, bicicletas, animales, diversidad de assets y optimización. El desarrollo real concentró el esfuerzo en construir una base sólida de IA vehicular, debido a la dificultad técnica de lograr navegación autónoma estable en la ciudad.

Durante el desarrollo se utilizaron asistentes de inteligencia artificial como apoyo de análisis y depuración. Su papel fue generar hipótesis, ordenar alternativas, proponer comprobaciones y ayudar a interpretar logs, Blueprints y CSV. Las respuestas no se incorporaron de forma automática: las decisiones se contrastaron con el comportamiento observado en Unreal Engine, las descripciones fechadas del programa, los resultados exportados y las pruebas de regresión. El anexo B documenta una selección depurada de estos prompts.

El seguimiento operativo se registró en GitLab mediante issues y milestones. El CSV depurado contiene 40 issues cerradas agrupadas en 15 sprints, desde el 7 de noviembre de 2025 hasta el 1 de julio de 2026. Esta fuente no sustituye a las actas de reunión ni a los resultados experimentales, pero sí aporta una trazabilidad intermedia: permite relacionar fechas, tareas cerradas y cambios técnicos concretos.

5.2 PLANIFICACIÓN TEMPORAL

La planificación real del proyecto puede entenderse como un diagrama de Gantt incremental. Las primeras semanas estuvieron dedicadas a comprender el proyecto heredado, ordenar assets y crear una primera señalización funcional. Después se concentró el esfuerzo en el vehículo autónomo, el entrenamiento evolutivo y el entorno de pruebas. A partir de marzo el trabajo se desplazó hacia la instrumentación, la separación entre entrenamiento y test, las intersecciones, la señalización avanzada y, finalmente, la interacción física entre vehículos.

Tabla 5.1. Diagrama de Gantt simplificado del desarrollo.

Fase	Periodo	Esf.	Resultado principal
Arranque y limpieza del proyecto	Nov.-dic.	■	Carpeta CarSimulator, assets ordenados y revisión del estado heredado.
Señalización y vehículo inicial	Dic.-ene.	■	Semáforos, señales verticales, primer coche con sensores y salidas continuas.
Línea de pruebas y aprendizaje básico	Ene.-feb.	■	Entrenamiento de centrado, corrección de desviación, curvas y primeros checkpoints.
Spawns, bordes e instrumentación	Feb.-mar.	■	Puntos de aparición distribuidos, bordes virtuales y primeros CSV de diagnóstico.
Train/test y señalización en la red	Mar.-abr.	■	Separación de modos, contexto one-hot, zonas de espera y cruce.
Fitness avanzado y persistencia	Abr.-may.	■	Penalizaciones, modelos persistentes, elitismo y mutación dinámica.
Rotondas y análisis de regresión	Jun.	■	Métricas por salida, balances de recompensa y test reproducibles.
Interacción física entre vehículos	Jun.-jul.	■	Solapes, distancia de seguridad, cedas con cola y simulación libre con respawn.

Fuente. Elaboración propia a partir de actas, milestones, issues y descripciones fechadas.

5.3 SEGUIMIENTO EN GITLAB

La tabla 5.2 resume las milestones registradas en GitLab. Se han agrupado por sprint para evitar una lista excesiva de issues individuales y para conservar la lectura global de la evolución.

Tabla 5.2. Milestones e issues cerradas registradas en GitLab.

Sprint	Periodo	Issues	Resultado documentado
0	07/11 a 27/11/2025	2	Reorganización inicial de Content, creación de CarSimulator y limpieza de assets heredados o importados.
1	27/11 a 11/12/2025	3	Familiarización con la IA existente, semáforo dinámico y primeras señales verticales con zonas de detección y parada.
2	11/12 a 24/12/2025	3	Primer vehículo autónomo con red inicial, mapeo de sensores y definición preliminar de perfiles de conducción con ASPAYM.
3	25/12/2025 a 08/01/2026	2	Milestone redefinida como <i>Gestor evolutivo, persistencia y restricciones de conducción</i> . Agrupa el sistema evolutivo de evaluación/guardado y las restricciones de velocidad, dirección, marcha atrás y centrado en carril.
4	08/01 a 16/01/2026	2	Milestone redefinida como <i>Validación inicial del modelo y documentación de prácticas</i> . Agrupa los primeros ciclos train/test sobre desviación y carril junto con la memoria de prácticas externas.
5	16/01 a 29/01/2026	2	Ajuste de la red para mantener carril y entrenamiento inicial en curvas en ambos sentidos.
6	18/02 a 04/03/2026	4	Bordes virtuales de intersección, mayor variedad de spawns, análisis de datos y multiraytracing.
7	05/03 a 19/03/2026	4	Modo test, exportación ampliada de datos, normalización del fitness por spawn y mejora de puntos de aparición.

Fuente. Elaboración propia a partir del CSV de issues y milestones de GitLab.

Tabla 5.2. Milestones e issues cerradas registradas en GitLab (continuación).

Sprint	Periodo	Issues	Resultado documentado
8	24/03 a 06/04/2026	4	Integración de señales y semáforos en la red, zonas de espera, zonas de cruce y modo <i>train/test</i> .
9	07/04 a 17/04/2026	3	Telemetría ampliada, entradas <i>one-hot</i> para contexto de tráfico y validación más segura de cruces.
10	20/04 a 30/04/2026	3	Fitness avanzado, tiempo de gracia tras paradas legales, estadísticas de éxito y ampliación de la capa oculta.
11	01/05 a 15/05/2026	2	Tasa de mutación dinámica, elitismo visual y refinamiento de la validación de paradas con tiempos aprendidos.
12	15/05 a 01/06/2026	2	Penalización por colisión direccional, detección de NAV_VIOLATION y ajuste de marcha atrás.
13	01/06 a 20/06/2026	2	Lógica de rotondas, sensores de entrada y salida, conteo de giros y telemetría específica por salida.
14	22/06 a 01/07/2026	2	Detección de solapes entre vehículos, distancia de seguridad y colas dinámicas de liberación en cedas.

Fuente. Elaboración propia a partir del CSV de issues y milestones de GitLab.

5.4 CRONOLOGÍA DE DECISIONES

La tabla 5.3 sintetiza los principales hitos recogidos en las actas de seguimiento.

Tabla 5.3. Principales hitos y decisiones documentadas en reuniones.

Fecha	Situación revisada	Decisión y consecuencia
27/11/2025	Integración de tráfico y elección de la primera IA.	Se prioriza la IA vehicular y se revisan semáforos dinámicos. Se descarta una malla dinámica para reconocer caminos por su coste de desarrollo.
11/12/2025	Representación de las rutas de la ciudad.	Se descarta mapear todo el escenario con splines y se prueba el <i>Path Finding</i> de Unreal. Se separan la decisión del modelo y la aplicación física del control.
17/12/2025	Interfaz entre la red y el vehículo.	Se fijan dos salidas continuas, dirección y aceleración/frenado, ambas dentro de $[-1, 1]$.
08/01/2026	Rigidez observada con <i>Path Finding</i> .	Se abandona esta opción como solución principal y se adopta entrenamiento evolutivo por prueba y error para aprender las maniobras básicas.
05/02/2026	Fallos en curvas e intersecciones.	Se introduce de forma provisional el etiquetado dinámico de bordes y se estudia cómo coordinar la elección de trayectoria con la percepción local.
19/02/2026	Falta de una validación conjunta de los subsistemas.	Se programa una demostración con red neuronal, sensores y bordes activos en todo el mapa.
05/03/2026	Demostración fallida y resultados difíciles de interpretar.	Se decide instrumentar la función de fitness y simplificar temporalmente los escenarios más complejos para localizar el origen de los fallos.
19/03/2026	Segunda demostración con navegación ya estable en buena parte del mapa.	Se acepta la base técnica, se planifica la reorganización del repositorio y se preparan la compilación para Windows y las siguientes mejoras.

Fuente. Elaboración propia a partir de las actas de seguimiento.

5.5 LÍNEA EVOLUTIVA DEL DESARROLLO

Además de los hitos de reunión, el proyecto ha seguido una evolución técnica marcada por pruebas que no siempre dieron un resultado utilizable a la primera. Esa evolución es relevante porque explica por qué la solución actual no se limita a una red neuronal aislada, sino que integra sensores, reglas de tráfico, persistencia de modelos, análisis de CSV y pruebas de regresión.

La tabla 5.4 resume las fases principales. No pretende listar cada cambio menor de Blueprint, sino recoger las decisiones que modificaron la dirección del trabajo.

Tabla 5.4. Línea evolutiva del desarrollo técnico.

Periodo	Prueba o problema	Aprendizaje incorporado
Noviembre-diciembre	Tráfico autónomo inicial y conexión red-vehículo.	Se comprobó que el objetivo debía centrarse primero en IA vehicular. La red quedó acoplada a dos salidas continuas, dirección y aceleración/frenado, para respetar la física del vehículo.
Enero-febrero	Rigidez de NavMesh y <i>Path Finding</i> .	Las rutas deterministas ayudaban a desplazar actores, pero no resolvían bien curvas, cruces ni control continuo. Se adoptó percepción local con sensores y aprendizaje evolutivo.
Marzo	Demostración fallida e interpretación insuficiente de los fallos.	La observación visual dejó de ser suficiente. Se instrumentó el fitness, se exportaron métricas por coche y se simplificaron temporalmente escenarios para localizar causas.
Marzo-abril	Precisión de parada, marcha atrás y coches inmóviles.	Se introdujeron penalizaciones específicas para parada imprecisa, retroceso y estancamiento injustificado. Las consultas de apoyo insistieron en evitar constantes arbitrarias y en derivar umbrales de variables ya presentes en el simulador.
Abril-mayo	Señalización, stops, cedas e intersecciones.	Los actores de tráfico pasaron a comunicar contexto al controlador. Los bordes de cruce se asociaron a dirección y trigger para que cada maniobra tuviera límites sensoriales coherentes.

Fuente. Elaboración propia a partir de actas, issues de GitLab, descripciones de Blueprints y análisis de datos.

Tabla 5.4. Línea evolutiva del desarrollo técnico (continuación).

Periodo	Prueba o problema	Aprendizaje incorporado
Mayo-junio	Persistencia de modelos, duplicados y selección para reentrenar.	Se revisó la estrategia de <code>SuperCarModel</code> , <code>SessionCarModel</code> y copias fechadas. También se compararon respaldos para evitar repetir tests sobre modelos equivalentes y para elegir candidatos de reentrenamiento con datos, no solo por fitness máximo.
Junio-julio	Separación train/test, rotondas y análisis de regresión.	Se diferenciaron spawns y trayectorias por modo, se añadieron métricas de rotonda, solapes entre vehículos, bloqueo en cruce y colas dinámicas de liberación en cedas. La serie final muestra que una mejora local del fitness puede degradar otra parte del sistema, por lo que cada cambio debe validarse con bloques comparables antes de aceptarse.

Fuente. Elaboración propia a partir de actas, issues de *GitLab*, descripciones de *Blueprints* y análisis de datos.

5.6 ENTORNO DE PRUEBAS INICIAL

Una parte decisiva del desarrollo fue construir una línea de pruebas pequeña antes de intentar que los vehículos circularan por toda la ciudad. Esa zona inicial reducía deliberadamente el problema: un mismo punto de aparición, carril reconocible, bordes controlados, checkpoints simples y una función de fitness centrada en avanzar, mantenerse dentro del carril, corregir desviaciones y aprender a girar. El objetivo no era representar todavía todo el tráfico urbano, sino aislar los conceptos mínimos que un coche físico necesitaba dominar antes de enfrentarse a intersecciones y señales.

Esta decisión evitó mezclar demasiadas causas de fallo en la misma prueba. Cuando el coche no giraba o se salía del carril, la explicación estaba en sensores, red, suavizado o fitness, no en una rotonda mal delimitada o en un stop situado demasiado cerca del spawn. Por eso la línea inicial funcionó como un banco de ensayo: permitió comprobar la conexión entre entradas, pesos, salidas y físicas de *Chaos Vehicle*, y también obligó a abandonar fórmulas de fitness que premiaban estrategias visualmente pobres.

El salto posterior a puntos de aparición dinámicos alrededor del mapa cambió el problema de escala. El gestor dejó de evaluar una habilidad local y pasó a distribuir la población entre zonas heterogéneas, con curvas, cruces, semáforos, stops y cedas.

La introducción de SpawnBag, marcas ValidForTrain/ValidForTest y normalización por spawn convirtió el entrenamiento en un proceso mucho más representativo, pero también más exigente: una mejora en un punto fácil podía dejar de ser dominante si no se sostenía en puntos de mayor dificultad. Ese cambio revolucionó el manager evolutivo porque obligó a seleccionar modelos por robustez espacial, no solo por el máximo de fitness alcanzado en una línea controlada.

5.7 ALCANCE

El alcance funcional completado se centra en conseguir que los vehículos de tráfico puedan aprender y circular dentro del escenario existente. El trabajo no modifica el vehículo conducido por el paciente ni rediseña la ciudad: actúa sobre los coches autónomos, su controlador y los elementos del mapa que les proporcionan información. El resultado es un controlador capaz de transformar las salidas de la red neuronal en dirección, aceleración y frenado sobre el sistema físico de Unreal Engine.

Para alimentar ese controlador se ha incorporado percepción local mediante ray-casting y se han normalizado las variables que entran en la red. El aprendizaje se realiza con poblaciones de vehículos, selección, mutación y elitismo. Los pesos obtenidos pueden guardarse y cargarse en sesiones posteriores, lo que evita reiniciar el proceso cada vez que se abre el simulador. La evaluación se apoya en una función de fitness que combina avance, centrado, velocidad, cumplimiento de señales y penalizaciones por comportamientos no deseados.

Incluye además semáforos, stops, cedas, líneas de parada y validación de maniobras. En cruces y rotondas se registran trayectorias, entrada, salida, giro, colisiones y recompensa, con puntos de aparición separados para entrenamiento y test. Los CSV resultantes permiten comparar ejecuciones y localizar fallos repetidos por zona.

No forman parte de esta versión la implementación completa de peatones, bicicletas y animales, ni los perfiles de conductor diferenciados previstos en la propuesta inicial. Tampoco se aborda todavía una validación con pacientes. Estas ampliaciones requieren primero consolidar la navegación vehicular y disponer de un protocolo de prueba estable, por lo que se mantienen como trabajo posterior.

5.8 COMPLEJIDAD DEL CAMBIO DE MAPA

El cambio o ampliación de mapa no fue un simple problema de que la red neuronal tuviera que aprender curvas nuevas. La dificultad temporal más importante estuvo en preparar manualmente la geometría útil para la percepción: colocar bordes de calzada, volúmenes de navegación, triggers de intersección, cajas de espera, zonas de cruce, flechas de parada y propiedades específicas de cada actor. Si un borde quedaba

asociado a la dirección equivocada, si un volumen se solapaba con una zona válida o si una línea de stop no estaba bien orientada, el coche recibía una lectura incoherente aunque la política neuronal fuese razonable.

Esta carga de configuración explica por qué algunas regresiones se concentraron en actores concretos, como bordes de intersección o `NavModifierVolume`. La red podía responder de forma aceptable ante una curva o rotonda nueva cuando los sensores recibían límites consistentes; en cambio, una propiedad mal asignada en el escenario convertía el entrenamiento en ruido. Por ello, el trabajo de mapa debe considerarse parte del sistema de IA: define qué percibe el agente, qué cuenta como salida de calzada, qué trayectoria está habilitada y cuándo una espera se valida como legal. En futuras ampliaciones convendría automatizar parte de esta configuración o crear herramientas de verificación visual antes de lanzar entrenamientos largos.

5.9 GESTIÓN DE RIESGOS

La tabla 5.5 relaciona los riesgos más relevantes con su impacto, probabilidad y estrategia de mitigación.

Tabla 5.5. Riesgos principales del proyecto.

Riesgo	Impacto	Prob.	Mitigación
Rigidez de navegación determinista	Comportamiento poco realista y baja utilidad clínica	Alta	Pivotar a percepción local con sensores y red neuronal.
Fitness mal calibrado	Aprendizaje de estrategias no deseadas	Alta	Instrumentar componentes, exportar CSV y analizar penalizaciones por familia de muerte.
Coste computacional	Entrenamientos lentos y dependencia de GPU/CPU	Media	Poblaciones moderadas, análisis offline y propuesta futura de modo sin renderizado.
Complejidad de intersecciones	Colisiones y violaciones normativas en cruces	Alta	Bordes dinámicos, zonas de cruce, validación de stops/cedas y separación train/test.

Fuente. *Elaboración propia a partir del seguimiento técnico del proyecto.*

Tabla 5.5. Riesgos principales del proyecto (continuación).

Riesgo	Impacto	Prob.	Mitigación
Configuración manual del mapa	Lecturas sensoriales incoherentes y regresiones localizadas	Alta	Revisar propiedades de bordes, volúmenes, triggers y líneas de parada antes de cada entrenamiento largo; documentar hotspots por actor.
Sobreajuste a spawns concretos	Buen rendimiento local pero mala generalización	Media	Normalización por punto de aparición y test con spawns diferenciados.
Pérdida de conocimiento entrenado	Reinicio de aprendizaje entre sesiones	Media	Persistencia de modelos SuperCarModel y SessionCarModel.

Fuente. Elaboración propia a partir del seguimiento técnico del proyecto.

5.10 ESTIMACIÓN DE RECURSOS Y PRESUPUESTO

El proyecto se ha desarrollado durante las prácticas y el TFG, sin contratación externa ni compra específica de material. El recurso principal ha sido el tiempo dedicado a implementar Blueprints, ejecutar entrenamientos, revisar resultados y documentar cambios. Parte de ese tiempo no es programación directa: los entrenamientos ocupan el equipo durante periodos prolongados y obligan a repetir pruebas cuando cambia el fitness o la geometría de una intersección.

El trabajo técnico se ha realizado en un equipo capaz de ejecutar Unreal Engine 5 y las físicas de varios vehículos de forma simultánea. El control de versiones se ha gestionado con GitLab y el análisis con Python y bibliotecas libres. Unreal Engine no ha generado coste de licencia en este contexto académico. Por tanto, no existen costes directos de software ni de infraestructura; el hardware ya estaba disponible y solo correspondería imputar amortización y consumo eléctrico si se elaborase un presupuesto comercial.

En la memoria final, la valoración económica deberá convertir las horas efectivas de desarrollo y experimentación a un coste de ingeniería acordado con el tutor, y añadir la parte proporcional del equipo utilizado. Mantener separadas ambas partidas evita presentar como gasto real una licencia no pagada y permite actualizar el presupuesto cuando se cierre el registro definitivo de dedicación.

Capítulo 6

Núcleo del trabajo

6.1 DESCRIPCIÓN GENERAL DE LA SOLUCIÓN

La solución actual se estructura como un sistema de agentes autónomos entrenados dentro del simulador. Cada vehículo dispone de un controlador de IA que percibe el entorno, ejecuta una red neuronal, aplica acciones al sistema físico y registra métricas de comportamiento. Por encima de los vehículos existe un gestor evolutivo que crea poblaciones, asigna pesos, evalúa resultados, selecciona los mejores individuos y guarda modelos.

El sistema se completa con elementos de tráfico que comunican contexto a los controladores: semáforos, señales de stop, señales de ceda y zonas de cruce. Estos elementos no son meros objetos visuales, sino actores con lógica propia que activan zonas de detección, informan al vehículo de líneas de parada, controlan estados y aplican penalizaciones o bonus según el comportamiento observado.

6.2 SUBSISTEMAS

6.2.1 Gestor evolutivo

BP_EvolutionManager coordina el ciclo de entrenamiento y las ejecuciones de test. Al comenzar comprueba si existe un modelo histórico en SuperCarModel. Si lo encuentra, carga sus pesos; en caso contrario, inicializa una red aleatoria que servirá como punto de partida. Después actualiza la lista de puntos de aparición permitidos para el modo activo y crea la población de vehículos, asignando a cada coche su controlador y su red.

En la primera generación, el gestor parte de redes aleatorias o del modelo guardado según la configuración. A partir de la segunda, el índice del coche determina su familia: se conserva una copia sin mutar del mejor modelo histórico y otra del mejor modelo de la sesión; un 40 % de los puestos disponibles se muta desde cada una de esas dos semillas con una tasa del 3 %; los puestos restantes se derivan del modelo histórico con una tasa del 6 %. Cada coche registra el número de pesos modificados, su

magnitud media y el fitness de su progenitor, lo que permite comprobar después si una familia de mutación aporta mejora real.

La generación termina cuando todos los vehículos han finalizado o han sido eliminados por alguna condición de muerte. El gestor ordena entonces los resultados y persiste el mejor de sesión en `SessionCarModel`. Solo sustituye `SuperCarModel` cuando el candidato mejora el fitness histórico y sus pesos son distintos; antes de sobrescribirlo guarda además una copia fechada del modelo anterior. La función `SaveWeightsToSlot` centraliza este proceso para que ambos ficheros almacenen los mismos bloques de pesos y el fitness asociado.

El modo de test reutiliza la misma infraestructura, pero desactiva la mutación y carga un modelo fijo. También emplea un conjunto de puntos de aparición distinto del utilizado para entrenar. Esta separación evita que una mejora aparente proceda únicamente de evaluar el coche en los recorridos que ya han condicionado su aprendizaje. Al terminar cada bloque, el gestor exporta tanto el resumen de la ejecución como los registros individuales necesarios para el análisis.

Una decisión relevante es la normalización por punto de aparición. La ciudad contiene zonas de dificultad desigual y comparar directamente el fitness de dos spawns puede favorecer al que comienza en una ruta sencilla. Por ello se conserva una referencia para cada punto y el resultado de cada coche se divide por la correspondiente antes de seleccionar sus pesos. Desde la versión del 17 de junio, esas referencias decaen en cada generación con un factor dependiente del historial, en lugar de conservar indefinidamente un máximo antiguo. La normalización no elimina las diferencias del mapa, pero evita que una ejecución excepcional bloquee de forma permanente la selección posterior.

En las últimas iteraciones, el gestor añadió un modo de simulación libre. Cuando `IsFreeRunMode` está activo, se genera un coche por punto de aparición válido, se carga el modelo consolidado, se desactiva el límite de vida y se comprueba periódicamente si los spawns quedan libres para respawnear vehículos muertos. Esta modalidad no sustituye al test estadístico, porque no produce generaciones cerradas comparables, pero permite observar tráfico continuo y localizar problemas de convivencia entre coches.

6.2.2 Transición a coches físicos no fantasma

Las primeras generaciones utilizaron una lógica equivalente a coches “fantasma”: cada individuo se evaluaba principalmente contra el mapa, los bordes y la señalización, sin que la interacción física con otros vehículos dominara el aprendizaje. Esa simplificación era razonable en las primeras fases, porque permitía entrenar carril, giro, freno y respuesta a señales sin mezclar el problema con colas, bloqueos o distancias de seguridad.

La fase final introdujo vehículos con colisiones y solapes reales entre sí. El controlador pasó a detectar coches visibles mediante las trazas, almacenar el vehículo más cercano y ajustar la distancia efectiva de obstáculo con la velocidad relativa. Además, BP_VehicleAI2 penaliza invasiones de espacio mediante NumCarOverlaps, CarOverlapPenaltyShadow y, desde julio, CarOverlapPenalty aplicado al fitness. Los BP_SpawnPoint incorporan una zona FreeCheckZone para impedir respawns inmediatos sobre vehículos activos.

Este cambio no se plantea como una corrección tardía, sino como una mejora progresiva del realismo. Primero se necesitaba una política capaz de circular de forma individual; después, al añadir tráfico físicamente concurrente, el sistema comenzó a medir distancia de seguridad, espera en cola, bloqueo de cruce y liberación dinámica de cedas. El coste fue una mayor complejidad experimental: un fallo podía proceder ya de la red, del fitness, de la geometría o de la interacción con otro coche. La ventaja es que los resultados finales se acercan más al objetivo real del simulador, que no es tener vehículos aislados, sino tráfico autónomo capaz de crear situaciones dinámicas para el usuario.

6.2.3 Controlador de IA y red neuronal

BP_AiController actúa como intermediario entre red neuronal y vehículo físico. En cada tick comprueba que el vehículo controlado existe y no está muerto, prepara las entradas de red, ejecuta la inferencia y aplica las salidas al componente de movimiento.

Para evitar saltos bruscos, el controlador dispone de variables de suavizado como MaxSteeringRate y MaxThrottleRate. La versión documentada mantiene un seguimiento estable entre objetivo y entrada aplicada, con gap medio cero en los informes finales, lo que indica que la señal de control se está transmitiendo sin divergencias instrumentales.

La inferencia se calcula manualmente sobre arrays de pesos. Primero, cada una de las 16 neuronas ocultas combina las 20 entradas normalizadas con su bloque de pesos y aplica $\tanh$. Después, las dos salidas repiten la operación sobre esas 16 activaciones ocultas, de nuevo con $\tanh$, para producir dirección y aceleración o frenado. La operación puede expresarse como:

$$h_j = \tanh \left(\sum_{i=1}^{20} W_{j,i}^{(1)} x_i \right), \quad j = 1, \dots, 16. \quad (6.1)$$

$$y_k = \tanh \left(\sum_{j=1}^{16} W_{k,j}^{(2)} h_j \right), \quad k = 1, 2. \quad (6.2)$$

La ecuación 6.1 calcula la activación de cada neurona oculta a partir del vector de entrada. La ecuación 6.2 obtiene después las dos salidas de control a partir de esas

activaciones. En estas expresiones, i recorre las entradas, j recorre las neuronas ocultas y k recorre las dos salidas. Por tanto, el uso de índices distintos sí está justificado: cada uno pertenece a una dimensión diferente de la red. En la segunda ecuación, j vuelve a aparecer como índice de suma porque las salidas se calculan a partir de todas las activaciones ocultas.

IH significa *Input to Hidden* y corresponde a las conexiones entre la entrada y la capa oculta. HO significa *Hidden to Output*, se denomina `WeightsIH` en los Blueprints y corresponde a las conexiones entre la capa oculta y las dos salidas. En las ecuaciones se ha usado $W^{(1)}$ y $W^{(2)}$ para que la notación matemática no dependa del nombre interno de los arrays.

Los pesos se almacenan en arrays unidimensionales. Por esa razón, el número de entradas esperado se obtiene dividiendo la longitud de `WeightsIH` entre el número de neuronas ocultas. A continuación se comprueba que ese resultado coincide con el vector de 20 entradas construido por el controlador, que `WeightsIH` contiene 20 por 16, es decir, 320 valores, y que `WeightsHO` contiene 16 por 2, es decir, 32 valores. Este cálculo dinámico permite detectar un modelo antiguo aunque la arquitectura vuelva a cambiar. Si faltan pesos o las dimensiones no son coherentes, se activa una guarda; después de cinco errores consecutivos, el coche se elimina con la razón de muerte `INVALID_BRAIN` para evitar que continúe circulando con una red inválida.

6.2.4 Percepción mediante sensores

El controlador lanza once trazas en abanico sobre 180 grados. Cada sensor devuelve una distancia normalizada hasta el primer elemento relevante. Los bordes de intersección se tratan con lógica específica: un borde virtual solo interviene en la lectura cuando su intención de dirección y su trigger coinciden con la decisión vigente del vehículo. Los bordes asociados a otras trayectorias se ignoran, de modo que una misma intersección puede contener límites sensoriales distintos para cada maniobra.

La decisión se modela mediante la enumeración `E_DirectionIntent`, con tres alternativas: primera, segunda y tercera salida. Al entrar en el trigger correspondiente, el vehículo selecciona aleatoriamente una de las opciones habilitadas para el modo de entrenamiento o de test. El controlador conserva tanto la dirección elegida como el trigger que la originó. Cada borde de intersección declara una o varias parejas formadas por dirección y trigger, y admite, cuando procede, cualquier trigger o cualquier dirección. De este modo, los rayos perciben únicamente los límites que delimitan la trayectoria seleccionada sin duplicar actores para los casos compartidos.

Las entradas sensoriales se combinan con información de tráfico. Si el vehículo entra en la zona de un semáforo, stop o ceda, el actor correspondiente comunica al controlador el tipo de control, la línea de parada y la distancia de entrada. Esta mezcla de

percepción geométrica y contexto normativo permite que la red no dependa únicamente de distancia a obstáculos.

6.2.5 Sistema de fitness

La función de fitness convierte lo que ocurre durante la conducción en una puntuación que el gestor evolutivo puede comparar. En este proyecto no se planteó como una fórmula única y cerrada, sino como una suma acumulativa de pequeñas decisiones: avanzar de forma útil aumenta la puntuación, mientras que circular sin espacio, corregir mal, incumplir una señal o quedar bloqueado la reduce. Esta elección fue práctica. En Unreal Engine el coche no se mueve sobre una línea ideal, sino con físicas, inercia, rozamiento, colisiones y sensores que pueden cambiar de lectura en cada cruce.

La primera versión era deliberadamente simple. En la línea de pruebas inicial bastaba con premiar avance, centrado y supervivencia para comprobar si la red podía aprender a mantenerse dentro del carril y tomar curvas básicas. Al llevar el sistema a la ciudad completa, esa puntuación dejó de ser suficiente: algunos coches sobrevivían sin progresar, otros compensaban una colisión con puntos acumulados y otros aceleraban en zonas donde había una señal o un obstáculo cercano. La evolución del fitness consistió precisamente en separar esos casos para que el modelo no aprendiera atajos visualmente aceptables pero inútiles para tráfico real.

Tabla 6.1. Lectura práctica de los cálculos principales del fitness.

Cálculo	Qué observa	Por qué importa
Avance útil	Velocidad hacia delante, aceleración aplicada y espacio disponible.	Selecciona coches que progresan de verdad y evita premiar supervivencia pasiva.
Prudencia frontal	Distancia al obstáculo más cercano, incluido otro vehículo cuando existe.	Reduce la tendencia a acelerar contra bordes, cruces o coches próximos.
Centrado y corrección	Diferencia entre sensores laterales y sentido de giro aplicado.	Premia correcciones coherentes y penaliza girar hacia el lado que aumenta la desviación.
Señalización	Contexto de semáforo, stop o ceda, distancia a la línea y estado del cruce.	Hace que frenar, esperar y reanudar la marcha formen parte del aprendizaje.
Seguridad física	Marcha atrás, colisión, salida de calzada, estancamiento y solapes.	Evita que el algoritmo conserve individuos peligrosos solo porque acumularon puntos antes del fallo.
Rotondas y tráfico	Entrada, salida, velocidad, acelerador, espera en cola y bloqueo en cruce.	Permite diagnosticar maniobras urbanas complejas sin esconderlas dentro del fitness final.

Fuente. *Elaboración propia a partir de BP_VehicleAI2, BP_Sign_Stop, BP_Sign_Yield y BP_IntersectionTrigger.*

El avance útil es el cálculo base. El coche recibe recompensa cuando se mueve hacia delante con una velocidad razonable y con suficiente espacio libre. La distancia frontal actúa como moderador: a mayor cercanía de un obstáculo, menor sentido tiene seguir acelerando. En las últimas versiones esta distancia no solo procede de paredes o bordes, sino también del coche más cercano detectado por los sensores. Así, la red deja de aprender una conducción aislada y empieza a recibir presión para respetar distancia de seguridad.

La prudencia frontal se introdujo porque el avance por sí solo generaba comportamientos agresivos. Si el coche circula demasiado rápido para el espacio que tiene delante, el fitness resta puntuación antes de que se produzca una colisión. Esto cambió la señal de aprendizaje: no se espera a que el fallo sea terminal, sino que se avisa al modelo durante la aproximación. La corrección de junio hizo este término más interpretable al evitar que la distancia frontal quedase sobredimensionada respecto a otros componentes.

El centrado se calcula comparando sensores laterales. Si la diferencia entre ambos lados es grande, el coche probablemente está cerca de un borde o desalineado respecto a la zona válida. Lo importante no es buscar un centro geométrico perfecto, porque en cruces y rotondas la trayectoria puede cambiar, sino comprobar si la dirección elegida ayuda a corregir la desviación. Cuando el volante actúa en sentido contrario al necesario, el sistema penaliza esa corrección errónea y registra el acumulado para poder revisarlo en los CSV.

La señalización se añadió después, cuando el vehículo ya era capaz de circular de forma básica. Semáforos, stops y cedas comunican al controlador el tipo de control, la línea de parada y el estado de la zona de cruce. El fitness distingue tres situaciones: aproximarse demasiado rápido a una obligación de parada, esperar correctamente y reanudar la marcha cuando el cruce queda libre. Los bonus por maniobra correcta se exportan por separado para que no oculten una infracción grave. Esta separación fue clave al analizar las regresiones por `STOP_VIOLATION`, porque permitió diferenciar cruzar la línea, agotar el tiempo de validación o quedarse bloqueado esperando.

Las penalizaciones de seguridad recogen los casos que no deben ser seleccionados aunque el coche haya acumulado puntos previamente. La marcha atrás se considera una señal de pérdida de control salvo en situaciones muy justificadas. La salida de calzada se acumula durante el tiempo que el vehículo permanece fuera de la zona válida. El estancamiento se evalúa comprobando si el coche apenas se desplaza y no existe una razón de tráfico que lo explique. Las colisiones terminan la ejecución con una razón de muerte concreta, y los solapes entre coches, incorporados al pasar a vehículos no fantasma, funcionan como aviso de convivencia física deficiente.

Las rotondas y los cedas con cola necesitaron registros propios. En rotonda se mide si el coche entra, cuánto tiempo permanece, si acelera de forma excesiva y si llega a completar la salida prevista. En ceda se distingue entre continuar porque el cruce está libre, esperar por otro vehículo y quedar bloqueado demasiado tiempo. Estos cálculos no buscan inflar el fitness final, sino explicar por qué una maniobra urbana falla. En los resultados se observa precisamente que una escala mal equilibrada puede mejorar una parte de la conducción y degradar otra, por lo que la utilidad de estas métricas está en guiar pruebas de regresión.

El resultado final es un fitness más largo de implementar, pero más claro de interpretar. Al terminar una ejecución no se conserva solo un número final: se exportan acumulados de avance, velocidad, frenado, espera, dirección, señalización, rotondas, navegación fuera de calzada, solapes, bloqueo en cruce y causa de finalización. Gracias a ese desglose puede saberse si un modelo condujo mejor o si simplemente sobrevivió más tiempo en una zona poco exigente. La principal limitación sigue siendo el equilibrio entre componentes. Cada ajuste cambia la presión de selección completa y por eso debe validarse con bloques comparables antes de aceptarse como mejora estable.

6.2.6 Semáforos

El sistema de semáforos se compone de un controlador de intersección y de actores de semáforo. El controlador alterna fases norte-sur y este-oeste, incluyendo verde, ámbar y todo rojo. Cada semáforo mantiene materiales dinámicos para representar estado visual y una zona de sensor que detecta vehículos.

Cuando un vehículo entra en la zona, el semáforo comunica estado, línea de parada y distancia. Si el semáforo está en rojo o ámbar restrictivo, el controlador activa contexto de parada. Al salir de la zona, se comprueba si el vehículo atravesó la línea a una velocidad legal. El sistema aprende umbrales de velocidad legal a partir de muestras previas, evitando depender siempre de un valor fijo.

6.2.7 Stop y ceda el paso

Las señales BP_Sign_Stop y BP_Sign_Yield amplían la interacción normativa. En stop se exige detención suficiente dentro de una ventana dinámica antes de la línea, espera mínima y comprobación de cruce libre. Si se cumple, se aplica bonus y se libera el vehículo; si no, se aplica penalización y muerte por STOP_VIOLATION. En ceda se monitoriza velocidad, aproximación y disponibilidad de la zona de cruce, con penalización por YIELD_VIOLATION y bonus al completar correctamente. La lógica más reciente diferencia, además, los cedas donde el vehículo puede continuar porque el cruce está libre de los casos en los que queda retenido por otros coches.

La liberación de cedas se ha refinado con una cola específica de vehículos pendientes. Cuando el cruce deja de estar ocupado, el sistema no libera todos los coches de forma inmediata ni depende de un único chequeo antiguo; añade el vehículo a PendingReleaseYieldCars y lo reevalúa mediante Recheck_YieldRelease cada 0.3 segundos. Si el cruce continúa libre y el identificador de comprobación sigue siendo válido, se aplica el bonus de ceda y se limpia el contexto. Si vuelve a haber coches en la zona de cruce, el vehículo regresa a la cola de espera. Este diseño reduce bloqueos innecesarios y deja registrada la espera acumulada para poder analizar regresiones.

La diferencia entre stop y ceda es metodológicamente importante. El stop requiere detención explícita; el ceda permite continuar si la velocidad y la prioridad son compatibles con la seguridad del cruce.

La validación del stop aprende de las maniobras legales una velocidad máxima y un tiempo mínimo de espera. Con ellos calcula una ventana de parada a partir de la distancia de entrada y del tiempo estimado hasta la línea. La revisión de final de junio simplifica el límite de validación a la suma del tiempo aprendido y el tiempo de aproximación. Como se muestra en el capítulo de resultados, este cambio coincide con un aumento acusado de `STOP_VIOLATION` y queda pendiente de recalibración. La separación posterior entre `STOP_VIOLATION_EXIT` y `STOP_VIOLATION_TIMEOUT` ayuda a distinguir si el coche cruza indebidamente la línea o si queda atrapado esperando demasiado tiempo.

6.2.8 Rotondas

Los triggers de intersección pueden marcarse como entrada o tramo de rotonda. Al entrar se registra la dirección escogida y se penaliza una velocidad superior a la aconsejada por la distancia frontal. Una nueva llegada a la entrada confirma que se ha completado una vuelta y aplica una bonificación proporcional a velocidad y espacio disponible. Durante el recorrido se acumulan giro, aceleración, colisiones y tiempo para la primera, segunda y tercera o posteriores salidas.

El fitness incluye además una penalización cuando el acelerador supera la distancia frontal normalizada dentro de la rotonda. En la revisión de final de junio, su escala se normaliza con el cociente entre el cuadrado de la velocidad máxima y la distancia máxima de visión, evitando que dependa de unidades incompatibles. Las recompensas de entrada y completado y las penalizaciones de aproximación y aceleración se exportan por separado; así puede comprobarse si el saldo neto favorece realmente completar la maniobra. Las descripciones posteriores añaden el análisis por salida, lo que permite detectar si la primera, la segunda o las salidas posteriores concentran una parte desproporcionada de los fallos.

6.3 PIPELINE EXPERIMENTAL

Cada ejecución genera un registro por vehículo y un resumen por generación. Una vez terminada la sesión, los scripts de Python validan que ambos CSV contienen el número de filas esperado y que sus identificadores pueden alinearse. Solo después de esa comprobación se calculan los indicadores y se generan las gráficas. De esta manera, una fila incompleta o una generación duplicada no queda oculta dentro de una media agregada.

El diagnóstico comienza con *BestFit*, *MeanFit* y tiempo de vida, pero no termina en esas tres cifras. Las razones de muerte se agrupan por familias para separar colisiones, finalizaciones por tiempo y fallos normativos. Los bonus de stop y ceda se contrastan con el contexto de frenado, y los resultados se desglosan por punto de aparición. También se revisan correlaciones entre componentes, genealogía de mutaciones y métricas de rotonda por salida. El analizador excluye de los promedios situacionales los coches que no estuvieron expuestos al evento, lo que evita interpretar un cero por ausencia de rotonda como una maniobra correcta.

Este procedimiento convierte los datos en una guía de implementación. Cuando un indicador empeora, se consulta el CSV individual para localizar actores, intersecciones o spawns concretos. Después se ajusta el Blueprint afectado y se repite la prueba. Así el análisis cierra el ciclo entre implementación, simulación y revisión del resultado, sin depender únicamente de lo observado en pantalla.

Capítulo 7

Resultados

7.1 DATOS ANALIZADOS

La evaluación principal se desplaza a la configuración estable del 18 de junio de 2026. El entrenamiento `train_2026-06-18_11-21` contiene 548 generaciones completas en `Training_Summary.csv` y 16432 registros en `Fitness_Debug.csv`. La columna `N` declara 16440 vehículos, por lo que la cobertura del fichero de detalle es del 99.95 %. El informe automático no detecta alertas fuertes de desalineación, pero esta diferencia de ocho filas se conserva como limitación del conjunto.

La evaluación de test agrega las cuatro carpetas del 18 de junio: 12-41, 14-03, 14-39 y 14-51. En conjunto reúnen 21 ejecuciones completas y 1134 vehículos, con 21 observaciones para cada uno de los 54 puntos de aparición. Los cuatro informes presentan cobertura del 100 %, no descartan filas y clasifican todos los registros como test. La hora sirve únicamente para identificar cada carpeta; no se considera una variable experimental.

Los tamaños efectivos se obtienen de los CSV: 30 vehículos por generación de entrenamiento y 54 por ejecución de test. Esta comprobación es necesaria porque `PopulationSize` es editable en la instancia del mapa y su valor por defecto no describe necesariamente la sesión archivada. También se detectó que las carpetas de test `2026-06-11_11-55` y `2026-06-11_13-08` contienen un `Fitness_Debug.csv` idéntico; en las comparaciones se cuenta una sola vez.

Los bloques posteriores de final de junio y comienzo de julio se emplean como pruebas de regresión y diagnóstico, no como nueva referencia principal. En esa serie aparecen recuperaciones parciales, caídas por `STOP_VIOLATION`, sesiones demasiado cortas y nuevos indicadores sobre cedas, solapes y bloqueo en cruces. El 2 de julio alcanza 193 aciertos sobre 354 evaluaciones, un 54.52 %, pero procede de un único bloque más pequeño que la referencia del día 18. El 3 de julio cambia además el modo experimental: se activa `FreeRun`, con coches vivos censurados y respawn por spawn-point, por lo que debe interpretarse como simulación libre de tráfico y no como test estadístico generacional. Por ello, los resultados principales del capítulo se apoyan en

la configuración validada del 18 de junio y las sesiones posteriores se interpretan como evidencia para decidir las siguientes correcciones.

7.2 RESUMEN CUANTITATIVO

Tabla 7.1. Resultados principales del entrenamiento y del bloque de test del 18 de junio de 2026.

Métrica	Entrenamiento	Test agregado
Generaciones o ejecuciones	548	21
Registros de vehículos	16432	1134
<i>BestFit</i> máximo	134959.45	80104.07
<i>MeanFit</i> medio	30049.20	44619.47
<i>MeanFit</i> mediano	29922.54	45313.89
Tiempo medio de vida	74.98 s	77.91 s
Ejecuciones con <i>MeanFit</i> negativo	0.00 %	0.00 %
Colisiones	45.54 %	40.04 %
Fallos legales	2.12 %	2.29 %
LAZY	0.00 %	0.00 %
Tasa de acierto	No aplica	57.50 %

Fuente. Elaboración propia a partir de los CSV del proyecto.

El test no mejora el modelo durante la ejecución: carga pesos fijos y desactiva la mutación. Su mayor *MeanFit* responde a que cambia la distribución de puntos y trayectorias y a que cada ejecución cubre los 54 spawns de test. Por ello, la tabla 7.1 describe cada modo, pero no permite atribuir la diferencia de fitness únicamente a la calidad de la política.

7.3 EVOLUCIÓN DEL ENTRENAMIENTO

El mejor fitness se alcanza en la generación 56, con 134959.45, y el fitness medio máximo en la generación 33, con 44373.00. Ninguna de las 548 generaciones presenta *MeanFit* negativo.

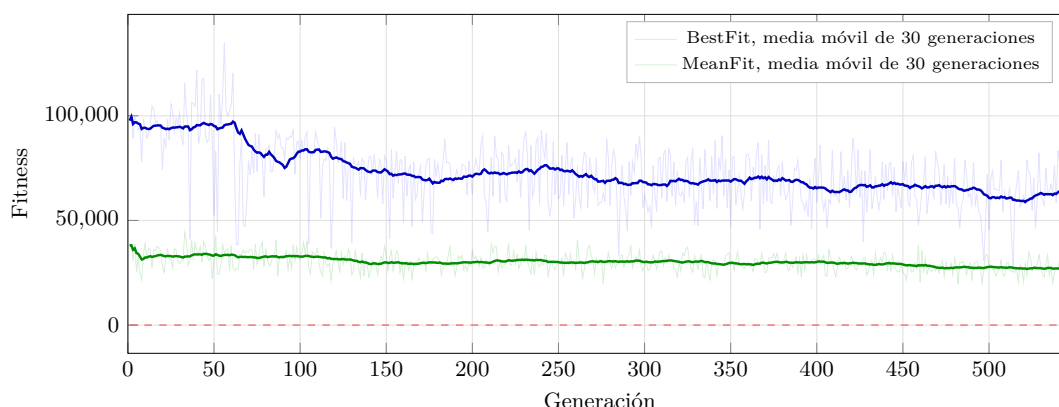
La figura 7.1 muestra variabilidad persistente y obliga a separar supervivencia de aptitud. Entre las primeras y las últimas 50 generaciones, el *MeanFit* desciende un 17.5 %, de 33263.23 a 27433.45, mientras el tiempo medio aumenta un 43.1 %, de 54.83 a 78.46 segundos. El informe no identifica convergencia clara y registra 492 generaciones desde el último máximo. El modelo aprende a permanecer activo durante más tiempo, pero esa supervivencia no se traduce en una mejora monotonía del objetivo.

De los 16432 registros, 8511 terminan por tiempo y 7483 por colisión. Los 348 fallos legales representan el 2.12 %, y no aparece ninguna muerte LAZY. El analizador

Tabla 7.2. Evolución de la red neuronal y del fitness durante el proyecto.

Periodo	Arquitectura y entradas	Evolución del fitness
Enero	11 sensores, velocidad, velocidad angular y bias; 12 neuronas ocultas.	Función centrada en avance, centrado lateral, giro y castigos simples por marcha atrás, colisión o estancamiento. Adecuada para la línea de pruebas inicial, insuficiente para ciudad completa.
Febrero-marzo	Sensores en abanico y primeros bordes de intersección filtrados por trayectoria.	El fitness deja de premiar solo avance y empieza a distinguir velocidad segura, corrección lateral, retroceso y estancamiento. El cambio a múltiples spawns obliga a normalizar por zona.
Abril	Vector ampliado con contexto de tráfico, distancia a línea de parada y codificación one-hot; transición hacia 20 entradas.	El fitness incorpora semáforos, stops, cedas, espera legal, marcha atrás, suavizado y telemetría ampliada. Aparecen regresiones que ya pueden explicarse con CSV.
Mayo-junio	Capa oculta consolidada en 16 neuronas; 320 pesos IH y 32 pesos HO.	Se estabilizan persistencia, mutación dinámica, normalización por spawn y métricas de rotonda. El 18 de junio se obtiene la referencia principal de test.
Junio-julio	Misma red, pero con percepción de otros coches, distancia efectiva y modo libre.	El fitness añade solapes, penalización real de distancia de seguridad, colas de ceda, bloqueo en cruce y diagnóstico de coches físicos no fantasma.

Fuente. Elaboración propia a partir de descripciones fechadas de REDNEURONAL, FITNESS y EVOLUTIONMANAGER.

Figura 7.1. Tendencia del fitness en el entrenamiento del 18 de junio de 2026.

Fuente. *Análisis generado a partir de Training_Summary.csv.*

marca 1985 posibles atascos ante stop y 1011 ante ceda dentro de las finalizaciones por tiempo. Son candidatos heurísticos, no infracciones confirmadas, pero explican por qué el tiempo de vida no puede utilizarse como único indicador.

La sesión registra además 5365 entradas en rotonda, 1075 completaciones confirmadas por una nueva entrada y 2350 colisiones dentro de ese contexto. La tasa de completado del 20.04 % es un límite inferior porque el contador solo confirma la maniobra al alcanzar de nuevo el trigger de entrada. El saldo específico de rotonda es negativo, con $-553966,78$ puntos acumulados, debido principalmente a la penalización de aceleración.

7.4 EVALUACIÓN DE TEST

El criterio implementado considera fallo cualquier colisión, infracción, estancamiento, corrección o marcha atrás incorrecta, vuelco, red inválida o navegación fuera de zona; también exige al menos 60 segundos de vida. El umbral de fitness de 25000 está configurado, pero desactivado mediante `TestUseFitness`. Con estas reglas, el bloque del 18 de junio obtiene 652 aciertos de 1134 evaluaciones, un 57.50 %. El intervalo de confianza de Wilson al 95 % se sitúa entre el 54.60 % y el 60.34 % [12]. Las cuatro sesiones oscilan entre el 53.70 % y el 61.42 %, con una mediana del 57.04 %.

La tabla 7.3 resume la secuencia de pruebas archivadas. No constituye un experimento controlado entre fechas, porque durante junio cambian el fitness, las intersecciones, el número de spawns y los modelos. Se utiliza para mostrar la evolución observada, no para atribuir causalidad a una modificación aislada. En esta tabla, el fitness medio se calcula directamente sobre `Fitness_Final`; por ello puede diferir ligeramente del promedio de *MeanFit* de la tabla 7.1. El duplicado exacto del 11 de junio se excluye. Cuando el resumen declara más evaluaciones que registros completos de detalle, se

conserva la cifra de detalle para el fitness y la vida media, y la tasa de acierto procede de `Test_Summary.csv`.

Tabla 7.3. Evolución de los bloques de test archivados entre junio y julio de 2026.

Fecha	Evaluaciones	Acierto	Fitness medio	Vida media
2 de junio	1975	16.51 %	27195.46	46.16 s
5 de junio	2080	18.51 %	26599.02	44.98 s
8 de junio	1560	25.19 %	31295.04	53.13 s
11 de junio	1092	29.67 %	33200.70	52.90 s
15 de junio	431	28.01 %	32758.50	53.90 s
16 de junio	1890	26.51 %	34104.90	55.10 s
17 de junio	540	39.63 %	40305.20	67.50 s
18 de junio	1134	57.50 %	44631.86	77.91 s
22 de junio	550	52.91 %	44296.62	76.20 s
23 de junio	1094	16.27 %	28264.74	50.21 s
26 de junio	467	49.89 %	41869.89	73.34 s
29 de junio	1179	30.42 %	33191.28	61.07 s
30 de junio	236	46.61 %	32663.97	75.06 s
1 de julio	767	27.38 %	17578.78	56.23 s
2 de julio	354	54.52 %	30276.78	80.27 s

Fuente. Elaboración propia a partir de `Fitness_Debug.csv` y `Test_Summary.csv`.

Frente al 8 de junio, el bloque del día 18 sube 32.31 puntos de acierto, un 42.6 % el fitness final medio y un 46.6 % la vida media. Como la serie cae el día 15 y en varias sesiones del 16, las mejoras se aceptan solo tras repeticiones.

Los bloques posteriores muestran por qué no basta con seguir la fecha más reciente. La serie pasa por una fase próxima a la referencia, una degradación normativa fuerte, una recuperación desigual y una mejora parcial en los dos tests del 30 de junio. Este último bloque reúne solo 236 evaluaciones válidas, por lo que no tiene el peso estadístico del día 18, pero sí confirma que el comportamiento debe valorarse por familias de fallo y no por una única cifra agregada. Ese contraste convierte las sesiones posteriores en material de diagnóstico, no en sustitución directa del bloque de referencia.

7.5 RAZONES DE FINALIZACIÓN

Las 652 finalizaciones por tiempo coinciden con los aciertos declarados en el resumen de test. Entre los 482 fallos hay 454 colisiones, 23 infracciones de stop, tres infracciones de tráfico y dos casos de marcha atrás. No aparecen LAZY, YIELD_VIOLATION, NAV_VIOLATION ni INVALID_BRAIN.

El actor con más colisiones es BP_IntersectionBorder221, con 94 casos. Le siguen NavModifierVolume43, con 64, y NavModifierVolume35, con 63. Los tres reúnen 221 fallos, el 48.7 % de todas las colisiones del bloque. La concentración mantiene como prioridad la revisión geométrica de esos elementos y de los recorridos que conducen hasta ellos.

7.6 COMPORTAMIENTO ANTE SEÑALIZACIÓN

En contexto de parada, el 99.34 % del uso acumulado de los pedales corresponde al freno y el 0.66 % al acelerador. Los registros contabilizan 622 stops y 1692 cedas completados; un mismo vehículo puede superar varias señales, por lo que estas cifras miden eventos y no coches únicos. Las 26 infracciones legales suponen el 2.29 % del bloque.

Estos datos confirman que la señalización modifica la acción del controlador, pero no demuestran por sí solos que toda espera sea correcta. La gran diferencia entre completaciones y muertes legales convive con finalizaciones por tiempo que el detector heurístico puede clasificar como atasco. Para validar una señal concreta siguen siendo necesarias la progresión de ruta, la duración continua de la detención y la inspección del spawn asociado.

7.7 COMPORTAMIENTO EN ROTONDAS

El bloque de test registra 443 entradas en rotonda, 31 completaciones confirmadas y 149 colisiones. Esto equivale a un mínimo del 7.00 % de completaciones y a 33.63 colisiones por cada 100 entradas. El saldo del componente específico es $-187295,02$: las bonificaciones de completado suman 82883.48 puntos, frente a 2408.29 de penalización de entrada y 267770.21 de penalización por aceleración.

El signo negativo no significa necesariamente que el vehículo nunca complete la maniobra, sino que la escala del castigo por acelerar domina sobre la recompensa. La corrección de unidades introducida en las iteraciones posteriores intenta resolver este desequilibrio. Los tests de seguimiento elevan en algunos casos el porcentaje mínimo de completaciones de rotonda, pero no estabilizan el comportamiento global: cuando mejora una parte de la maniobra, los fallos legales o los conflictos de geometría vuelven a dominar el resultado. Por tanto, la escala de rotonda parece mejor instrumentada, pero no puede validarse de forma aislada mientras el stop, el ceda y los cruces sigan introduciendo regresiones.

7.8 DIFICULTAD POR PUNTOS DE APARICIÓN

Cada uno de los 54 spawns dispone de 21 observaciones. Los puntos 16, 9 y 25 alcanzan los mayores fitness medios, con 68705.3, 68516.8 y 67472.3; sus tasas de acierto son 95.2 %, 100.0 % y 85.7 %. En el extremo opuesto, los puntos 3, 36 y 42 obtienen 10250.2, 14349.2 y 14722.7, con tasas del 9.5 %, 14.3 % y 19.0 %.

La diferencia entre el mejor y el peor fitness medio supera los 58000 puntos. Como todos tienen el mismo número de observaciones, no procede de un desequilibrio de muestreo. Puede deberse a la geometría inicial, la primera intersección, las trayectorias habilitadas o la adaptación del modelo a esa zona; por ello, los puntos 3, 36 y 42 deben utilizarse en pruebas dirigidas y los puntos 16, 9 y 25 como control de regresión.

7.9 PRUEBAS DE REGRESIÓN Y DIAGNÓSTICO

Tras el bloque de referencia se realizaron nuevas iteraciones sobre fitness, stops, cedas, rotondas e instrumentación. No forman un experimento controlado con una única variable modificada, por lo que su valor está en detectar qué problemas reaparecen y qué métricas permiten explicarlos. La primera fase conserva parte del comportamiento del 18 de junio, pero los entrenamientos asociados elevan los fallos legales por encima del 50 %, casi siempre por `STOP_VIOLATION`.

La regresión queda clara cuando el bloque del 23 de junio cae al 16.27 % de acierto y concentra casi la mitad de sus fallos en causas legales. Después aparecen ejecuciones muy dispares: una sesión corta no aporta suficiente detalle, otra alcanza un resultado favorable aislado y un bloque más amplio vuelve a bajar al 30.42 %. Esta variabilidad impide sustituir la referencia estable por una sesión posterior, aunque alguna cifra puntual parezca prometedora.

Las pruebas del 30 de junio añaden dos bloques pequeños de test con 236 evaluaciones válidas y un 46.61 % de acierto agregado. El resultado mejora respecto al 29 de junio, pero sigue por debajo de la referencia y procede de un tamaño menor. Su aportación principal es diagnóstica: distinguen `STOP_VIOLATION_TIMEOUT`, cedas liberados sin parada completa, colas de liberación en ceda, tiempo bloqueado en cruce, solapes entre coches y velocidad en la validación de ceda.

Los entrenamientos del 30 de junio y del 1 de julio refuerzan esa lectura. Ambos usan el nuevo esquema de métricas, pero continúan dominados por fallos legales; el segundo desplaza el problema hacia `STOP_VIOLATION_TIMEOUT` y detecta tiempo bloqueado en cruce en 6390 de 8872 coches seleccionados. Esto señala que la lógica de cruce, liberación y espera debe revisarse junto al fitness.

El 1 de julio agrupa cinco bloques de test con 767 evaluaciones declaradas y un 27.38 % de acierto. La causa dominante ya no es una única familia: conviven

TIMEFINISHED, STOP_VIOLATION_TIMEOUT, YIELD_VIOLATION_TIMEOUT y colisiones en volúmenes de navegación. El 2 de julio recupera parcialmente el comportamiento, con 193 aciertos sobre 354 evaluaciones y 54.52 % de éxito, aunque todavía queda por debajo del 57.50 % del 18 de junio y presenta 65 STOP_VIOLATION_EXIT. Por tanto, el día 2 se interpreta como recuperación parcial, no como nueva referencia cerrada.

La sesión del 3 de julio debe leerse aparte. El informe automático detecta FreeRun: 300 eventos terminales exportados, 50 vehículos vivos censurados estimados y unos 350 intentos totales. En los eventos terminales aparecen 203 muertes LAZY, 53 STOP_VIOLATION_EXIT, 7 YIELD_VIOLATION_EXIT y 37 colisiones. También se registran 419 solapes entre coches en 247 de los 300 vehículos terminales, 63399.08 puntos de penalización real por solape, 697.60 segundos acumulados de bloqueo en cruce y 122 entradas en rotonda con 6 completaciones confirmadas. Estos datos son valiosos para evaluar tráfico continuo y coches no fantasma, pero no deben compararse directamente con la tasa de acierto de un test por generaciones.

La conclusión experimental es que el bloque del 18 de junio sigue siendo la referencia de esta memoria. Mejora la tasa de acierto, reduce colisiones respecto a etapas previas y amplía la evaluación a 54 spawns. Aun así, el 40.04 % de colisiones, la diferencia entre puntos de aparición, el saldo negativo en rotondas y las regresiones posteriores mantienen abierto el problema. La versión estable debe definirse por un bloque reproducible de entrenamiento y test; las sesiones posteriores sirven para decidir la siguiente corrección, no para reemplazar la referencia por fecha.

Capítulo 8

Normativa y accesibilidad

8.1 PROTECCIÓN DE DATOS

El proyecto se orienta a una aplicación potencial en rehabilitación cognitiva, por lo que debe contemplar desde el diseño la protección de datos personales y, especialmente, de datos de salud. Durante el desarrollo descrito en esta memoria no se ha trabajado con datos reales de pacientes. Los registros analizados corresponden a vehículos autónomos simulados, métricas de entrenamiento y resultados técnicos.

En una futura validación clínica, cualquier uso con personas usuarias deberá ajustarse al Reglamento General de Protección de Datos y a la Ley Orgánica 3/2018 de Protección de Datos Personales y garantía de los derechos digitales [13, 14]. Será necesario definir responsable del tratamiento, finalidad, base jurídica, minimización de datos, conservación, consentimiento informado cuando proceda, medidas de seudonimización y control de acceso. También deberá diferenciarse entre datos necesarios para la evaluación terapéutica y métricas técnicas internas del simulador.

8.2 SEGURIDAD Y USO RESPONSABLE

El simulador no sustituye por sí mismo una evaluación clínica ni una autorización administrativa para conducir. Su función debe entenderse como apoyo a profesionales, entrenamiento controlado y herramienta de observación. Cualquier conclusión sobre capacidad real de conducción debería depender de especialistas y procedimientos establecidos.

Desde el punto de vista técnico, el sistema autónomo genera tráfico artificial. Por tanto, sus fallos no suponen riesgo físico directo durante el entrenamiento virtual, pero sí pueden afectar a la validez de la experiencia. Es importante que las sesiones con pacientes utilicen modelos suficientemente probados para evitar comportamientos incoherentes que confundan al usuario o introduzcan estímulos no deseados.

8.3 LICENCIAS DE ASSETS EXTERNOS

El proyecto utiliza una mesh de semáforo descargada de Fab, identificada en la documentación del trabajo mediante la URL <https://fab.com/s/c00389c4b9f2>. Fab distingue entre licencias Creative Commons Attribution para determinados assets gratuitos y licencia Standard para assets gratuitos o de pago [8]. En la licencia Standard existen los niveles Personal y Professional, que dependen de los ingresos brutos de actividad comercial en los últimos 12 meses, pero ambos conceden el mismo alcance de uso sobre el asset [7].

Según el resumen oficial de la Fab Standard License, el asset puede utilizarse de forma privada o comercial, modificarse para incorporarlo a un proyecto, utilizarse con herramientas compatibles no limitadas a Unreal Engine y distribuirse comercialmente integrado dentro del proyecto final [7]. También puede compartirse con colaboradores que trabajen en el mismo proyecto, por ejemplo mediante un repositorio privado. La restricción principal es que no puede revenderse ni redistribuirse como asset independiente, ni gratis ni de pago, ni facilitar que terceros lo hagan [7].

Para esta memoria, el uso realizado es compatible con esa finalidad: la mesh actúa como elemento visual integrado en el simulador y no como producto autónomo. Aunque Fab indica que no es obligatorio acreditar al publicador bajo la licencia Standard, se conserva la referencia al origen por trazabilidad académica y para facilitar una revisión posterior. Si en el futuro el simulador se distribuye fuera del entorno académico, deberá comprobarse de nuevo la ficha concreta del asset descargado, conservar la licencia aceptada en el momento de adquisición y evitar incluir el archivo fuente de la mesh en repositorios públicos donde pueda extraerse como recurso independiente.

8.4 ACCESIBILIDAD

La accesibilidad del simulador debe analizarse en relación con las personas a las que se dirige, que pueden presentar alteraciones de atención, memoria, percepción o movilidad. En este TFG no se ha rediseñado la interfaz utilizada por el paciente, por lo que no puede afirmarse que el producto completo haya sido validado como accesible. La aportación realizada se concentra en el tráfico autónomo y afecta de forma indirecta a la carga cognitiva de cada sesión.

El número de vehículos, los puntos del mapa y la frecuencia de situaciones normativas deben poder ajustarse para que la dificultad aumente de forma progresiva. También es importante repetir un mismo escenario bajo condiciones comparables: si el tráfico cambia sin control, resulta difícil saber si una variación en el resultado procede del paciente o de una sesión más exigente. La separación entre entrenamiento y test de la IA y el registro de eventos sientan una base técnica para construir esas repeticiones,

aunque todavía falta trasladar estos parámetros a una configuración manejable por el profesional.

Desde la perspectiva de accesibilidad, el tráfico autónomo debe comportarse como una variable graduable. Un perfil de baja dificultad debería reducir densidad, velocidad y número de decisiones simultáneas; un perfil avanzado podría introducir cruces ocupados, esperas en ceda, rotondas y vehículos próximos. Esta graduación no depende solo del modelo neuronal, sino también del gestor de spawns, del modo de ejecución y de los indicadores que se presenten al especialista. Por ello, la accesibilidad futura debe contemplar tanto la interfaz del paciente como las herramientas de configuración clínica.

La interacción debe contemplar periféricos adaptados, instrucciones breves y legibles y una presentación que no añada estímulos irrelevantes. El rendimiento gráfico y la duración de la prueba también deben vigilarse para reducir fatiga o mareo. Por último, los datos exportados no deberían mostrarse al especialista como un volcado técnico de CSV, sino como indicadores comprensibles y vinculados a maniobras concretas. Estos requisitos deberán revisarse con profesionales y usuarios de ASPAYM antes de utilizar el simulador en una validación clínica.

Capítulo 9

Conclusiones y líneas futuras

9.1 CONCLUSIONES

El punto de partida del trabajo era una ciudad preparada para la conducción del usuario, pero sin un sistema de tráfico capaz de recorrerla de forma autónoma. Las primeras pruebas confirmaron que trasladar soluciones habituales de navegación de personajes a un coche con físicas producía recorridos rígidos y problemas en los cruces. La decisión de sustituir las rutas cerradas por percepción local y una política neuronal permitió avanzar, aunque el resultado no debe interpretarse como un controlador ya resuelto.

La arquitectura implementada conecta once sensores, velocidad, giro y contexto de señalización con una red de 20 entradas, 16 neuronas ocultas y dos salidas. Sus 352 pesos se validan antes de cada inferencia y las consignas se aplican de forma suavizada al componente físico. El gestor evolutivo conserva por separado el modelo histórico y el de sesión, distribuye mutaciones de distinta intensidad y normaliza la selección por punto de aparición. Esta integración funciona dentro del escenario real del simulador y no en una prueba aislada, que era una de las dificultades principales del proyecto.

El desarrollo también ha puesto de manifiesto que la función de fitness y el registro de datos forman parte de la solución, no son tareas posteriores. Una demostración visual podía parecer correcta y ocultar estancamientos, penalizaciones mal equilibradas o resultados concentrados en unos pocos puntos del mapa. La exportación por vehículo permitió revisar esas situaciones y motivó cambios concretos en el fitness, en los bordes de intersección y en la separación entre entrenamiento y test.

Los cuatro tests del 18 de junio reúnen 1134 evaluaciones y alcanzan un 57.50 % de acierto, con un intervalo de confianza del 95 % entre el 54.60 % y el 60.34 %. Las colisiones descienden al 40.04 % y los fallos legales representan el 2.29 %. La mejora frente al bloque del día 8 es notable, pero el rendimiento sigue dependiendo del punto de aparición y las rotondas acumulan un saldo de fitness negativo. El entrenamiento asociado tampoco presenta convergencia clara: aumenta el tiempo de vida mientras desciende el fitness medio entre el inicio y el final de la sesión.

Las ejecuciones posteriores refuerzan esa cautela. Después de modificar la escala del fitness, la lógica de parada y el tratamiento de intersecciones, la serie alterna recuperaciones parciales con regresiones marcadas por `STOP_VIOLATION`. Algunas sesiones aisladas obtienen resultados aceptables, pero los bloques amplios vuelven a mostrar inestabilidad y diferencias importantes entre causas de fallo. Las últimas pruebas aportan métricas más finas sobre `STOP_VIOLATION_TIMEOUT`, cedas liberados, bloqueo en cruces, solapes entre coches y espera en colas dinámicas de ceda. Esta evolución se interpreta como una regresión detectada y mejor instrumentada, no como una nueva referencia. El resultado muestra el valor práctico de conservar configuraciones y de contrastar cada cambio con más de una métrica.

Por tanto, se ha obtenido una base funcional para generar, entrenar y medir tráfico autónomo, pero todavía no un sistema suficientemente robusto para una sesión clínica. La utilidad inmediata del trabajo reside en que los fallos ya pueden localizarse y compararse con un protocolo común. Las siguientes iteraciones deberán demostrar que las mejoras se mantienen en intersecciones y puntos de inicio distintos antes de aumentar la densidad de tráfico o incorporar nuevos tipos de agentes.

9.2 APORTACIONES REALIZADAS

La primera aportación es el controlador neuronal integrado en el vehículo de Unreal Engine. El trabajo incluye la construcción del vector de 20 entradas, la inferencia de la red y la aplicación suavizada de las dos salidas al sistema Chaos Vehicle. Los sensores no se limitan a detectar obstáculos: en las intersecciones filtran los bordes según la pareja de trigger y maniobra elegida y combinan esa geometría con el estado de semáforos, stops y cedas.

La segunda aportación es el proceso de aprendizaje dentro del propio simulador. El gestor crea poblaciones, conserva sin mutar las dos semillas principales, reparte mutaciones del 3% y del 6% y guarda por separado el mejor modelo histórico y el mejor de sesión. La referencia de cada spawn decae con el historial para evitar máximos permanentes, y el uso de puntos y opciones de trayectoria distintos en test permite evaluar el modelo fuera del conjunto que condicionó su entrenamiento.

La tercera aportación es la infraestructura de evaluación. Los Blueprints exportan el resultado de cada vehículo y los resúmenes de generación; posteriormente, los scripts comprueban la calidad de los CSV, agrupan razones de muerte y producen comparaciones por sesión, spawn, familia de mutación, solapes entre vehículos, bloqueo en cruce, validación y cola de ceda, y salida de rotonda. Esta conexión entre simulación y análisis permite justificar una modificación con datos y reconocer una regresión aunque el cambio local pareciera razonable.

Otra aportación relevante es la transición hacia tráfico físicamente concurrente. La incorporación de coches no fantasma, detección de solapes, penalización real por distancia de seguridad, zonas de respawn libres y simulación continua con FreeRun acerca el sistema al uso previsto: generar tráfico dinámico alrededor del usuario, no únicamente evaluar coches aislados contra el mapa. Esta mejora aumenta el realismo y también revela nuevas fuentes de fallo, como colas, bloqueos y conflictos entre vehículos.

Finalmente, las actas, las issues de GitLab y las descripciones fechadas conservan la evolución técnica del proyecto. Gracias a ellas puede reconstruirse por qué se abandonó *Path Finding*, cuándo se reformuló el fitness o cómo se introdujeron los bordes dependientes de la trayectoria. El registro depurado de 40 issues cerradas aporta una capa adicional de trazabilidad entre reuniones, implementación y resultados. Esto resulta especialmente útil en un desarrollo compartido que deberá continuar después del TFG.

9.3 PROBLEMAS ENCONTRADOS

El primer problema fue adaptar la navegación de Unreal Engine a un vehículo con inercia y radio de giro. NavMesh y *Path Finding* encontraban un destino, pero no producían una conducción natural. Abandonar esa vía obligó a trabajar conjuntamente con Chaos Vehicle, el control mediante Blueprints y los sensores de calzada.

Las intersecciones concentraron buena parte de los fallos posteriores. Un mismo cruce debía ofrecer límites distintos según la salida elegida, y esos límites tenían que coincidir con el trigger que había originado la decisión. La asociación explícita de parejas formadas por dirección y trigger resolvió parte del problema y permitió reutilizar bordes comunes. Aun así, BP_IntersectionBorder221 concentra 94 colisiones en el bloque de referencia, y las rotondas fallan con mayor frecuencia en recorridos que alcanzan salidas posteriores.

El ajuste del fitness fue otra dificultad importante. Premiar avance y supervivencia generó estrategias no previstas, entre ellas permanecer detenido o compensar una colisión con recompensas acumuladas. En rotondas ocurrió el problema contrario: la penalización de aceleración superó ampliamente las bonificaciones. La corrección posterior de la escala coincidió con una regresión masiva de stops, y las pruebas de seguimiento confirmaron que una ejecución favorable aislada no basta para cerrar el problema. La instrumentación aumentó la complejidad de los datos, pero evitó continuar ajustando el modelo únicamente a partir de la impresión visual.

Por último, cada entrenamiento exige ejecutar física y renderizado para una población completa. Este coste limita la velocidad con la que pueden probarse cambios y hace especialmente caro descubrir tarde un error de configuración. La persistencia de

modelos, la validación de los CSV y los bloques de test repetidos reducen ese riesgo, aunque sigue pendiente un modo de entrenamiento más rápido.

9.4 LÍNEAS FUTURAS

La siguiente iteración debería comenzar por los puntos 3, 36 y 42 y por tres actores concretos: el borde de intersección 221 y los volúmenes de navegación 43 y 35. En cada caso conviene separar un error de geometría de un fallo del controlador: revisar orientación y bordes, repetir la primera maniobra con el modelo del día 18 y comparar distancia recorrida, tiempo detenido y causa de finalización. Los puntos 16, 9 y 25 deben mantenerse como control para detectar regresiones en recorridos que ahora funcionan mejor.

Antes de un nuevo entrenamiento largo debe revertirse o recalibrarse el límite de validación de stop introducido en las últimas iteraciones. La prueba adecuada consiste en mantener el modelo, el fitness restante y los spawns, variar solo ese límite y ejecutar varios bloques cortos de regresión. Después seguirá siendo necesario distinguir una supervivencia útil de un coche inmóvil ante una señal mediante progreso de ruta, duración continua del atasco, tiempo bloqueado en cruce y subtipo de infracción. Los tests deberán conservar tamaño, modelo y conjunto de spawns para que los intervalos de confianza sean comparables.

También debe equilibrarse el término de rotonda. Las 31 completaciones confirmadas frente a 443 entradas y el saldo neto negativo aconsejan validar por separado la penalización de entrada, la de aceleración y el bonus de salida. Las salidas segunda y tercera requieren pruebas dirigidas, porque acumulan una proporción de muertes superior a la primera.

Para reducir el tiempo entre iteraciones se plantea un modo sin renderizado o con la simulación desacoplada de la presentación. Esta mejora permitiría entrenar más generaciones y reservar la ejecución completa para validar física, señalización y comportamiento visual. También sería conveniente automatizar pruebas sobre intersecciones concretas antes de lanzar una sesión larga.

Otra línea inmediata es integrar mejor la configuración del experimento con *VictoryPlugin*. El objetivo sería exportar e importar perfiles de ejecución con variables clave como tamaño de población, umbrales de test, tiempos de vida, modo train/test, activación de *FreeRun*, selección de spawns y parámetros de mutación. Con ello podrían ofrecerse perfiles comprensibles para distintos usos: un *Modo Ligero*, con menor población y ejecución más rápida para depuración, y un *Modo Completo*, con más spawns y métricas extensas para validación. Esta configuración reduciría errores manuales antes de entrenamientos largos.

También debe evolucionar el sistema de puntos de aparición. Actualmente los BP_SpawnPoint permiten separar train/test y comprobar si la zona está libre, pero siguen dependiendo de colocación manual. Una mejora consistiría en generar spawns autónomos, espaciados y orientados de forma coherente con el carril, ajustando automáticamente la cantidad a la población elegida. Esto permitiría que el mapa se adapte al experimento, evitaría concentraciones de coches en zonas concretas y facilitaría comparar sesiones con distintas densidades de tráfico.

En accesibilidad, el simulador debería incorporar ayudas para usuarios invidentes o con baja visión. Además de mejorar contraste, tamaño de elementos y claridad de instrucciones, se pueden estudiar estímulos hápticos mediante vibraciones y señales acústicas avanzadas, por ejemplo pitidos diferenciados para proximidad, semáforo, salida de carril o riesgo de colisión. La compatibilidad con volantes y pedales adaptados también es prioritaria si el simulador se orienta a rehabilitación y valoración funcional. Estas mejoras deberán diseñarse con profesionales y usuarios, no solo desde criterios técnicos.

Por último, el comportamiento de conducción avanzada queda abierto para fases posteriores. Una vez estabilizada la circulación básica, el tráfico autónomo debería incorporar cambio de carril, adelantamientos, incorporación a vías, gestión de prioridad entre varios coches y conducción anticipativa ante retenciones. Estas lógicas requieren que la red o el gestor dispongan de contexto adicional, porque no basta con reaccionar al obstáculo frontal: el vehículo debe decidir cuándo esperar, cuándo ceder, cuándo rebasar y cómo coordinarse con otros agentes.

Los perfiles de conductor normal, inseguro y agresivo, la interacción entre vehículos y la incorporación de peatones o bicicletas deben abordarse cuando la base vehicular sea estable. Añadir agentes antes de resolver las colisiones actuales dificultaría atribuir cada fallo a su causa. Finalmente, cualquier uso con pacientes requerirá definir con ASPAYM un protocolo de sesión, criterios de accesibilidad e indicadores comprensibles para los profesionales.

Agradecimientos

Pendiente de redacción definitiva.

Bibliografía

- [1] L. Fuente, *CognitiveCarSimulatorReloaded - IA*, propuesta de proyecto, 2025.
- [2] E. Romero Yuste, *CognitiveCarSimulator*, Trabajo de Fin de Grado, Grado en Desarrollo de Aplicaciones Interactivas 3D y Videojuegos, Escuela Politécnica Superior de Zamora, 2025.
- [3] Epic Games, *Unreal Engine 5 Documentation*, Epic Developer Community. Disponible en: documentación oficial de Unreal Engine (consulta: 8 de junio de 2026).
- [4] Epic Games, *Chaos Vehicles*, Epic Developer Community. Disponible en: documentación oficial de Chaos Vehicles (consulta: 8 de junio de 2026).
- [5] Epic Games, *Blueprints Visual Scripting in Unreal Engine*, Epic Developer Community. Disponible en: documentación oficial de Blueprints (consulta: 29 de junio de 2026).
- [6] Epic Games, *Traces with Raycasts*, Epic Developer Community. Disponible en: documentación oficial de trazas y raycasts (consulta: 29 de junio de 2026).
- [7] Epic Games, *Fab Standard License*, Fab. Disponible en: Fab End User License Agreement (consulta: 3 de julio de 2026).
- [8] Epic Games, *Licenses and Pricing in Fab*, Epic Developer Community. Disponible en: documentación oficial de licencias y precios de Fab (consulta: 3 de julio de 2026).
- [9] G. Cybenko, "Approximation by superpositions of a sigmoidal function", *Mathematics of Control, Signals and Systems*, vol. 2, pp. 303–314, 1989. DOI: 10.1007/BF02551274.
- [10] A. E. Eiben y J. E. Smith, *Introduction to Evolutionary Computing*, 2.^a ed., Springer, 2015. DOI: 10.1007/978-3-662-44874-8.
- [11] K. O. Stanley y R. Miikkulainen, "Evolving Neural Networks through Augmenting Topologies", *Evolutionary Computation*, vol. 10, n.º 2, pp. 99–127, 2002. DOI: 10.1162/106365602320169811.

- [12] E. B. Wilson, “Probable Inference, the Law of Succession, and Statistical Inference”, *Journal of the American Statistical Association*, vol. 22, n.º 158, pp. 209–212, 1927. DOI: 10.1080/01621459.1927.10502953.
- [13] Parlamento Europeo y Consejo de la Unión Europea, *Reglamento (UE) 2016/679, de 27 de abril de 2016, relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales y a la libre circulación de estos datos*, Diario Oficial de la Unión Europea, 2016. Disponible en: EUR-Lex (consulta: 8 de junio de 2026).
- [14] Jefatura del Estado, *Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y garantía de los derechos digitales*, Boletín Oficial del Estado, núm. 294, 6 de diciembre de 2018. Disponible en: BOE (consulta: 8 de junio de 2026).

Apéndice A

Fuentes analizadas

A.1 DOCUMENTACIÓN ADMINISTRATIVA Y DE FORMATO

Para la redacción de esta memoria se revisaron los documentos incluidos en INFO/UTILES. Esta documentación sirvió para fijar el formato de portada, comprobar los datos administrativos del trabajo y contrastar la descripción inicial del proyecto con el desarrollo realizado. Los documentos más relevantes fueron:

- portada y normas básicas de estilo de la Escuela de Ingenierías Industrial, Informática y Aeroespacial.
- solicitud de preinscripción del TFG, con título, descripción y tutoría.
- resolución favorable de preinscripción de fecha 18 de mayo de 2026.
- propuesta *CognitiveCarSimulatorReloaded* - IA.
- memoria previa *CognitiveCarSimulator*.
- informe de seguimiento de prácticas HP SCDS.
- memoria final de prácticas del alumno.
- estructura orientativa propuesta por el tutor.
- ficha y licencia del asset de semáforo descargado desde Fab.

A.2 ACTAS DE REUNIÓN

Se han analizado ocho actas de seguimiento, comprendidas entre el 27/11/2025 y el 19/03/2026. Estas actas permiten justificar las decisiones de arquitectura y los cambios de enfoque realizados durante el proyecto.

A.3 SEGUIMIENTO EN GITLAB

Se revisó el fichero `INFO/UTILES/gitlab_issues_final_limpio.csv`, exportado a partir de las issues y milestones del proyecto en GitLab. El CSV contiene 40 issues cerradas agrupadas en 15 sprints, entre el 7 de noviembre de 2025 y el 1 de julio de 2026. Para la memoria se depuraron títulos y descripciones, manteniendo fechas, identificadores, comentarios y enlaces. Esta fuente refuerza la planificación real, la línea evolutiva y la relación entre tareas cerradas y cambios técnicos.

A.4 DESCRIPCIONES DEL PROGRAMA

Las descripciones fechadas de los Blueprints se utilizaron para reconstruir el estado de la implementación sin depender únicamente de la versión abierta en el editor. Para el gestor se revisó la carpeta `EVOLUTIONMANAGER`, donde se documentan la creación de generaciones, la asignación de modelos, la mutación, la persistencia y el cambio al modo de test. La carpeta `REDNEURONAL` permitió comprobar la composición de las entradas, las dimensiones de `WeightsIH` y `WeightsHO`, la inferencia y las guardas frente a modelos incompatibles.

El cálculo de aptitud y su instrumentación proceden de `FITNESS`. Se revisaron versiones hasta el 3 de julio de 2026, incluidas las nuevas métricas de genealogía, rotondas, navegación fuera de calzada, solapes entre vehículos, bloqueo en cruces, bonus de espera en cola, cedas liberados y correcciones de escala de distancia frontal y aceleración. Para la señalización se consultó `INTERSECTIONS`, con la validación aprendida de semáforos y stops, opciones de trayectoria separadas por modo, parejas de dirección y trigger, lógica específica de rotondas, colas dinámicas de liberación en cedas y separación de subtipos de infracción. Por último, `OTROS` reúne macros auxiliares, trazas, detección de estancamiento y tareas pendientes que ayudan a interpretar dependencias entre módulos.

A.5 RESULTADOS EXPERIMENTALES

Los resultados documentados proceden de `INFO/ANALISIS_DATOS/Analisis`. La referencia principal de entrenamiento es `train_2026-06-18_11-21`, con 548 generaciones y 16432 registros de vehículos. La evaluación agrega las cuatro carpetas de test del 18 de junio, que suman 21 ejecuciones y 1134 registros sobre 54 spawns. La marca horaria forma parte del identificador y no se utiliza como variable experimental.

Para reconstruir la evolución se revisaron además los bloques de los días 2, 5, 8, 11, 15, 16, 17, 22, 23, 26, 29 y 30 de junio. Las carpetas `test_2026-06-11_11-55` y

test_2026-06-11_13-08 contienen exactamente el mismo CSV de detalle y se contabilizan una sola vez. El test del día 15 conserva 431 de las 432 filas declaradas. En las sesiones nuevas también se conserva visible la diferencia entre evaluaciones declaradas y registros de detalle cuando existe un pequeño descuadre; en test_2026-06-30_16-56, por ejemplo, el analizador selecciona la generación presente en Test_Summary.csv y excluye filas de una generación sin resumen cerrado. Esta incidencia y las coberturas de cada sesión se mantienen visibles en el capítulo de resultados.

También se analizaron los entrenamientos de los días 11, 16, 17, 18, 22, 23, 25, 26, 29 y 30 de junio, además de los entrenamientos del 1 y 2 de julio. Los bloques posteriores al día 18 no sustituyen a la referencia estable porque muestran regresiones normativas, entrenamientos incompletos o resultados demasiado variables entre sesiones. Se conservan para documentar el efecto de los cambios de fitness, stop, ceda, rotondas, diagnóstico de solapes y bloqueo en cruce. La sesión del 3 de julio se trató aparte como FreeRun, ya que exporta eventos terminales y coches vivos censurados en lugar de generaciones cerradas. La figura del capítulo 7 se ha regenerado en PDF vectorial a partir del Training_Summary.csv del día 18, y tanto los datos como el código PGFPlots quedan archivados en MEMORIA/assets.

A.6 CONVERSACIONES CON ASISTENTES DE IA

La carpeta INFO/CONVERSACIONES_IA contiene conversaciones mantenidas durante el desarrollo con ChatGPT, Claude y Gemini. Se han revisado 48 ficheros Markdown: 13 de ChatGPT, 28 de Claude y 7 de Gemini. En conjunto reúnen 769 bloques de consulta o intervención del usuario. No se han tratado como fuente primaria de resultados, sino como trazabilidad del proceso de trabajo: preguntas de depuración, análisis de entrenamientos, selección de modelos, organización del proyecto, uso de Unreal Engine y preparación de scripts auxiliares.

El criterio de incorporación ha sido selectivo. Las consultas menores sobre manejo del editor, avisos de LaTeX o dudas puntuales solo se han usado para entender el contexto. En cambio, sí se han tenido en cuenta las conversaciones que ayudan a explicar decisiones técnicas: precisión de parada, detección de INVALID_BRAIN, coches inmóviles sin justificación, separación train/test, duplicados de modelos, diagnóstico de rotondas, análisis de caídas de rendimiento y regresiones por STOP_VIOLATION. El anexo B recoge una versión corregida y mejorada de los prompts más representativos, junto con la comprobación realizada antes de aplicar cada idea.

Apéndice B

Prompts utilizados

B.1 CRITERIO DE DOCUMENTACIÓN

Este anexo documenta una selección representativa de los prompts utilizados durante el desarrollo. No se reproducen de forma literal: se presentan en una versión revisada, más clara y reutilizable, manteniendo el objetivo real de cada consulta. La decisión responde a dos motivos. Primero, muchos prompts originales formaban parte de conversaciones largas, con adjuntos, correcciones sucesivas y mensajes de seguimiento que no resultan legibles como evidencia académica. Segundo, el interés del anexo no es conservar cada frase escrita, sino explicar cómo se utilizó la IA generativa dentro del proceso de ingeniería.

Las respuestas de los asistentes se trataron como apoyo para generar hipótesis, ordenar alternativas y proponer comprobaciones. Ninguna decisión se incorporó únicamente por aparecer en una respuesta: se contrastó con Blueprints, CSV, informes de análisis, pruebas en Unreal Engine o revisión manual. Las consultas menores sobre uso básico del editor, avisos de LaTeX o formato no se incluyen salvo que ayudasen a reconstruir una decisión técnica.

B.2 MATERIAL REVISADO

La carpeta INFO/CONVERSACIONES_IA contiene 48 conversaciones exportadas en Markdown. La tabla B.1 resume el material revisado.

Tabla B.1. Conversaciones de IA revisadas para el anexo de prompts.

Asistente	Ficheros	Bloques	Uso principal
ChatGPT	13	209	Depuración de Blueprints, scripts de análisis, modelos y memoria.
Claude	28	292	Análisis de entrenamientos, regresiones, stops, rotondas y selección de modelos.
Gemini	7	268	Apoyo inicial con Unreal Engine, organización y consultas puntuales.

Fuente. Elaboración propia a partir de INFO/CONVERSACIONES_IA.

B.3 PROMPTS DEPURADOS INCORPORADOS

1. Prompt B.1. Análisis del estado del programa y mejora de parada

Conversación base. ChatGPT, mejora de parada precisa.

Objetivo. Mejorar la precisión de parada ante señales de stop y semáforos sin depender de una recompensa genérica de supervivencia.

Prompt depurado.

Analiza las descripciones actuales del controlador de IA, señales de stop, semáforos, fitness y gestor evolutivo. Quiero que el modelo aprenda a detenerse cerca de `StopTargetLocation` cuando exista obligación de parar. Identifica qué información ya recibe la red, qué parte falta en el fitness y qué cambios son necesarios. Evita constantes arbitrarias: si propones umbrales, indica de qué variable del simulador deben derivarse. Separa la propuesta en pasos verificables y añade qué métricas habría que exportar para comprobar si la mejora funciona.

Respuesta aprovechada. Se aprovechó la idea de distinguir entre tener información de parada como entrada y tener presión explícita en el fitness para aproximarse correctamente a la línea.

Comprobación realizada. Se revisaron las descripciones fechadas de red neuronal, fitness e intersecciones, además de los CSV con tiempos de espera y completaciones de stop.

Uso final en el proyecto. La memoria recoge la validación de stops, la ventana de parada, el aprendizaje de umbrales y la necesidad de diferenciar espera correcta de estancamiento.

2. Prompt B.2. Penalización de retroceso y estancamiento injustificado

Conversación base. Gemini, organización del proyecto, y ChatGPT, mejora de parada precisa.

Objetivo. Evitar que el entrenamiento premie coches que sobreviven sin avanzar, retroceden o permanecen inmóviles sin una causa de tráfico válida.

Prompt depurado.

Revisa la lógica de fitness y detección de muerte para vehículos que retroceden o se quedan parados. Necesito diferenciar una parada legítima, por semáforo, stop, ceda u obstáculo delante, de una parada que solo alarga la vida del coche sin aportar recorrido útil. Propón una lógica basada en velocidad real, contexto de tráfico, sensores frontales y tiempo de vida. No uses números fijos sin justificación. Indica qué razón de muerte o métrica de diagnóstico debe registrarse para validar el cambio.

Respuesta aprovechada. Se aprovechó la separación conceptual entre parada justificada y comportamiento perezoso, además de la necesidad de penalizar el retroceso con una causa trazable.

Comprobación realizada. Se comprobó con los informes de entrenamiento, las razones de muerte LAZY, BACKWARDS y TIMEFINISHED, y los casos de coches vivos sin progreso.

Uso final en el proyecto. El capítulo 6 describe la penalización por marcha atrás, estancamiento y navegación no válida, y el capítulo 7 advierte que el tiempo de vida no puede usarse como único indicador.

3. Prompt B.3. Diagnóstico de salida de calzada y coches bloqueados

Conversación base. Claude, análisis de problemas en datos de entrenamiento y comportamiento de vehículos.

Objetivo. Explicar por qué algunos coches fuera de calzada o retenidos en señales llegaban a finalizar por tiempo en lugar de morir por una causa clara.

Prompt depurado.

Analiza el entrenamiento adjunto y las descripciones de fitness, intersecciones, gestor evolutivo y red neuronal. Busca coches que sobrevivan demasiado tiempo fuera de calzada o parados ante stops y cedas. Distingue hipótesis de evidencia en los CSV. Calcula qué condiciones deberían activar NAV_VIOLATION, LAZY, STOP_VIOLATION o TIMEFINISHED. Propón soluciones dinámicas, basadas en variables existentes, y explica cómo verificarlas con métricas exportadas.

Respuesta aprovechada. Se aprovechó el análisis de interacción entre penalización acumulada, velocidad casi nula, contexto ForceStop y reinicio de contadores.

Comprobación realizada. Se contrastó con registros de navegación fuera de calzada, finalizaciones por tiempo, penalizaciones acumuladas y ausencia de muertes NAV_VIOLATION en determinadas sesiones.

Uso final en el proyecto. La memoria incorpora la salida de calzada como problema de acumulación temporal y de penalización, y no como simple colisión geométrica.

4. Prompt B.4. Actualización del analizador de CSV

Conversación base. ChatGPT, ajuste del código de análisis y gráficas de tiempo vivo.

Objetivo. Mantener los scripts de análisis alineados con las columnas nuevas exportadas por los Blueprints.

Prompt depurado.

Actualiza el script de análisis para los CSV actuales. Lee primero las cabeceras de Fitness_Debug.csv, Training_Summary.csv y Test_Summary.csv. Añade compatibilidad con las nuevas columnas de stop, ceda, suavizado de dirección, genealogía, rotondas y solapes entre vehículos. El script debe validar cobertura, duplicados, filas incompletas, distribución de razones de muerte, evolución de fitness, vida media y diferencias por spawn. Si una columna no existe en una sesión antigua, debe degradar con aviso y no fallar.

Respuesta aprovechada. Se aprovechó el enfoque de script robusto por cabeceras, con diagnóstico de cobertura y generación de informes comparables entre sesiones.

Comprobación realizada. Se ejecutaron análisis sobre sesiones de abril, mayo y junio, comprobando informes, gráficas y coherencia con los CSV originales.

Uso final en el proyecto. El capítulo 7 se apoya en métricas calculadas a partir de estos CSV y conserva explícitamente incidencias de cobertura o duplicados.

5. Prompt B.5. Separación de entrenamiento y test

Conversación base. ChatGPT, modificación de triggers y refresco de spawns.

Objetivo. Evitar que el rendimiento de test se midiera sobre los mismos recorridos que condicionaban el entrenamiento.

Prompt depurado.

Revisa la lógica de spawns y triggers de dirección. Quiero separar entrenamiento y test: en entrenamiento deben usarse solo los puntos y trayectorias habilitados para aprender, mientras que el test debe evaluar un conjunto independiente y completo. Indica qué variables deben pertenecer al trigger, cuáles al gestor evolutivo y en qué momento conviene refrescar la lista de spawns para evitar mezclas entre modos. Propón comprobaciones en CSV para verificar que cada ejecución usa el conjunto correcto.

Respuesta aprovechada. Se aprovechó la necesidad de separar TrainMode, listas válidas por modo, refresco de spawns y trayectorias permitidas.

Comprobación realizada. Se revisaron BP_Trigger, RefreshSpawnPoints, los resúmenes de test y la distribución de observaciones por los 54 spawns.

Uso final en el proyecto. La memoria presenta la separación train/test como decisión metodológica clave y advierte que el test no debe interpretarse como nueva fase de aprendizaje.

6. Prompt B.6. Selección y deduplicación de modelos

Conversación base. ChatGPT, duplicados de modelos, y Claude, selección de modelo para reentrenamiento.

Objetivo. Reducir pruebas repetidas sobre backups equivalentes y seleccionar candidatos de reentrenamiento con criterios trazables.

Prompt depurado.

Analiza los backups .sav de modelos y los tests asociados. Primero identifica modelos duplicados o casi idénticos comparando sus pesos, para no repetir evaluaciones. Después ordena los candidatos de reentrenamiento usando tasa de acierto, fitness medio, vida media, razones de muerte, estabilidad por spawn y coherencia entre entrenamiento y test. No elijas solo por *BestFit*. Explica qué modelo conviene conservar como base, cuál usar como alternativa y qué pruebas faltan antes de aceptar el cambio.

Respuesta aprovechada. Se aprovechó la idea de no tratar cada copia fechada como un modelo independiente si sus pesos no aportaban una política distinta.

Comprobación realizada. Se compararon pesos, informes de test, métricas por sesión y diferencias entre SuperCarModel y SessionCarModel.

Uso final en el proyecto. El capítulo 6 explica la persistencia de modelos y el capítulo 7 evita sustituir la referencia por sesiones posteriores no reproducibles.

7. Prompt B.7. Diagnóstico de rotondas e intersecciones

Conversación base. ChatGPT, revisión de comportamiento del coche y análisis de modelos.

Objetivo. Analizar por qué el vehículo fallaba en rotondas, especialmente en salidas posteriores, y cómo ajustar la geometría o el fitness sin perder generalización.

Prompt depurado.

Revisa el test del modelo y las descripciones de intersecciones. Me interesa entender el comportamiento en rotondas: entrada, permanencia, elección de salida, aceleración, colisiones y completado. Separa problemas de geometría, escala del fitness y percepción por sensores. Si propones mover puntos de spline o modificar bordes de intersección, justifica cómo afectará a la trayectoria y qué métrica permitirá verificar la mejora. Incluye una revisión específica para primera, segunda y tercera o posteriores salidas.

Respuesta aprovechada. Se aprovechó la distinción entre problema geométrico, problema de recompensa y problema de percepción contextual.

Comprobación realizada. Se revisaron métricas de entrada y completado de rotonda, colisiones, saldo específico de rotonda y análisis por salida.

Uso final en el proyecto. Los capítulos 6 y 7 describen las rotondas como subsistema propio, con penalización de aceleración, completaciones confirmadas y limitaciones de escala.

8. Prompt B.8. Análisis de regresión tras cambios de código

Conversación base. Claude, análisis de regresión y errores en stops, con atención a STOP_VIOLATION.

Objetivo. Entender por qué una modificación local podía empeorar el comportamiento global, especialmente en stops.

Prompt depurado.

Compara dos ejecuciones del mismo modelo antes y después de los cambios en intersecciones, fitness y red neuronal. Usa los informes y CSV para cuantificar la caída de rendimiento. Localiza qué razones de muerte aumentan, qué cambios de Blueprint coinciden temporalmente y qué hipótesis explican la regresión. No atribuyas causalidad sin prueba: separa correlación, evidencia directa y comprobaciones pendientes. Propón una secuencia de pruebas cortas en la que solo cambie un componente cada vez.

Respuesta aprovechada. Se aprovechó la metodología de comparar el mismo modelo bajo condiciones próximas, localizar cambios de Blueprints y mirar razones de muerte antes de aceptar una mejora.

Comprobación realizada. Se contrastaron los bloques de referencia y seguimiento de junio, con especial atención a STOP_VIOLATION, STOP_VIOLATION_TIMEOUT y a la variabilidad entre sesiones.

Uso final en el proyecto. La conclusión experimental indica que una mejora local del fitness puede degradar stops o generalización, por lo que la versión estable debe definirse por bloques reproducibles.

9. Prompt B.9. Mutación dinámica y persistencia de modelos

Conversación base. Claude, ratio dinámico basado en población, y ChatGPT, mejora del guardado de modelos.

Objetivo. Mejorar la creación de generaciones y el guardado de modelos sin depender de proporciones fijas ni duplicar copias equivalentes.

Prompt depurado.

Revisa `StartGeneration`, `EndGeneration` y `SaveWeightsToSlot`. Quiero que la población se reparta de forma dinámica según `PopulationSize`, conservando un individuo sin mutar del mejor modelo histórico y otro del mejor modelo de sesión. El resto debe distribuirse entre mutaciones y candidatos aleatorios o derivados, sin números mágicos. Además, revisa el criterio de guardado para evitar duplicados de pesos, pero sin perder un modelo que tenga fitness o metadatos relevantes. Explica qué campos deben registrarse para auditar la decisión.

Respuesta aprovechada. Se aprovechó el enfoque de ratios dependientes de población y la necesidad de guardar modelo, fitness y procedencia de forma coherente.

Comprobación realizada. Se revisaron las descripciones de `EVOLUTIONMANAGER`, los backups de modelos y las métricas de genealogía por índice de coche.

Uso final en el proyecto. La memoria describe las familias de mutación, el elitismo, `SessionCarModel`, `SuperCarModel` y la copia fechada del modelo anterior.

10. Prompt B.10. Organización del proyecto y reproducibilidad

Conversación base. Gemini, organización del proyecto, ChatGPT, limpieza de logs, y Gemini, errores de módulos en Unreal Engine.

Objetivo. Mantener el proyecto reproducible pese a trabajar con Blueprints, binarios, logs extensos y resultados experimentales pesados.

Prompt depurado.

Organiza el repositorio y el flujo de trabajo de un proyecto con Blueprints, modelos guardados, logs, CSV de entrenamiento y memoria en LaTeX. Distingue archivos versionables, temporales y evidencia experimental; propón nombres de carpetas para reconstruir una sesión y un procedimiento de limpieza que conserve warnings, mensajes propios de Blueprints y datos útiles de depuración.

Respuesta aprovechada. Se aprovechó la separación entre evidencia de proyecto, resultados generados, logs temporales y documentación final.

Comprobación realizada. Se revisó la estructura `INFO`, los scripts de análisis, los ficheros temporales de LaTeX y la limpieza de imágenes generadas no necesarias.

Uso final en el proyecto. La memoria conserva un anexo de fuentes, un anexo de prompts, reglas de limpieza del repositorio y una distinción entre datos primarios y salidas generadas.